BỘ KHOA HỌC VÀ CÔNG NGHỆ

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG




# BÁO CÁO ASSIGNMENT 05

## PHÁT TRIỂN HỆ THỐNG THÔNG MINH

**Giảng viên:** **Trần Đình Quế**

**Sinh viên:** **Nguyễn Tuấn Tài**

**Mã sinh viên:** **B23DCCN731**

**Nhóm lớp:** **06**

***Hà Nội – 2026***

# Mục lục

1. GIỚI THIỆU VÀ MỤC TIÊU ..... 5
1.1. Bối cảnh nghiên cứu học máy và mạng tích chập ..... 5
1.2. Mục tiêu kỹ thuật cốt lõi ..... 5
1.3. Ba tập dữ liệu thực nghiệm ..... 5
1.4. Quy trình thực nghiệm tổng thể ..... 6
2. CƠ SỞ LÝ THUYẾT VỀ MẠNG NƠ-RON VÀ CNN ..... 7
2.1. Bản chất hàm hợp của mạng nơ-ron sâu ..... 7
2.2. Trích xuất đặc trưng tự động và học biểu diễn ..... 8
2.3. Cấu trúc một lớp Convolutional ..... 8
2.4. Phép toán tích chập và bản chất nhân ma trận ..... 8
2.5. Kích thước Kernel ..... 8
2.6. Bước nhảy trượt ..... 9
2.7. Cơ chế đệm viền ..... 9
2.8. Vùng cảm nhận không gian ..... 9
2.9. Hàm kích hoạt phi tuyến ..... 9
2.10. Tầng gộp giảm chiều ..... 9
2.11. Tầng kết nối đầy đủ ..... 10
2.12. Lan truyền ngược và thuật toán cập nhật Adam ..... 10
3. SỰ PHÁT TRIỂN CỦA CÁC KIẾN TRÚC CNN KINH ĐIỂN ..... 10
3.1. LeNet-5 ..... 10
3.2. AlexNet ..... 10
3.3. VGG Network ..... 10
3.4. ResNet ..... 11
3.5. MobileNet và các kiến trúc hiện đại ..... 11
4. CƠ SỞ LÝ THUYẾT VỀ CÁC KỸ THUẬT CẢI TIẾN CNN ..... 11
4.1. Giới hạn cố hữu của Basic CNN ..... 11
4.2. Chuẩn hóa theo lô ..... 11
4.3. Kết nối phần dư ..... 12
4.4. Cơ chế chú ý kênh ..... 12
4.5. Phân tích so sánh chuỗi cấu trúc cải tiến M1 – M4 ..... 13
4.6. Mở rộng phép tích chập Conv1D cho dữ liệu dạng bảng ..... 13

4.7. So sánh các mô hình cổ điển thích nghi ..... 13
5. DỮ LIỆU VÀ BÀI TOÁN THỰC NGHIỆM ..... 13
5.1. Giới thiệu 3 bài toán nghiên cứu ..... 13
5.2. Tập dữ liệu MNIST ..... 14
5.3. Tập dữ liệu Fashion-MNIST ..... 14
5.4. Tập dữ liệu Bệnh tiểu đường Pima Indians Diabetes ..... 15
5.5. Chiến lược phân chia dữ liệu Train / Validation / Test ..... 17
5.6. Tiền xử lý dữ liệu và chuẩn hóa thang đo ..... 17
5.7. Xây dựng PyTorch DataLoader ..... 17
5.8. Bảng tổng kết dữ liệu thực nghiệm ..... 18
6. THIẾT KẾ VÀ CÀI ĐẶT CÁC MÔ HÌNH CNN ..... 18
6.1. Mô hình M1: Basic CNN ..... 18
6.2. Mô hình M2: BatchNorm CNN ..... 19
6.3. Mô hình M3: Residual CNN ..... 20
6.4. Mô hình M4: Attention CNN ..... 21
6.5. Mô hình M5: Adapted LeNet ..... 22
6.6. Mô hình M6: Adapted AlexNet ..... 22
6.7. Mô hình M7: Adapted VGG ..... 23
6.8. Thiết kế các mô hình Conv1D cho dữ liệu bảng Diabetes ..... 23
6.9. Kiểm tra Output Shape và Thống kê Không gian tham số ..... 26
7. QUY TRÌNH HUẤN LUYỆN VÀ ĐÁNH GIÁ ĐỐI CHUẨN ..... 27
7.1. Thiết lập siêu tham số và tối ưu hóa ..... 27
7.2. Cơ chế lưu trữ trạng thái tối ưu ..... 27
7.3. Đánh giá độc lập trên Test Set ..... 27
7.4. Hệ thống các độ đo đánh giá toàn diện ..... 27
8. KẾT QUẢ THỰC NGHIỆM ĐỐI CHUẨN ..... 28
8.1. Kết quả thực nghiệm trên tập dữ liệu MNIST ..... 28
8.2. Kết quả thực nghiệm trên tập dữ liệu Fashion-MNIST ..... 28
8.3. Kết quả thực nghiệm trên tập dữ liệu Diabetes ..... 29
8.4. Phân tích chi tiết các độ đo phân loại ..... 29
8.5. Mức độ cải thiện độ chính xác so với mô hình cơ sở M1 ..... 30
8.6. Phân tích độ phức tạp tính toán và chi phí tham số ..... 30

9. TRỰC QUAN HÓA VÀ PHÂN TÍCH BIỂU ĐỒ KHOA HỌC ..... 30
9.1. Vai trò của trực quan hóa trong nghiên cứu học sâu ..... 30
9.2. Phân tích đường cong Loss trên MNIST ..... 31
9.3. Phân tích đường cong Loss trên Fashion-MNIST ..... 31
9.4. Phân tích đường cong Loss trên Diabetes ..... 32
9.5. Phân tích so sánh Test Accuracy trên MNIST ..... 33
9.6. Phân tích so sánh Test Accuracy trên Fashion-MNIST ..... 33
9.7. Phân tích so sánh Test Accuracy trên Diabetes ..... 34
9.8. Phân tích thời gian huấn luyện ..... 35
9.9. Phân tích số tham số huấn luyện ..... 35
9.10. Phân tích Ma trận nhầm lẫn trên MNIST ..... 36
9.11. Phân tích Ma trận nhầm lẫn trên Fashion-MNIST ..... 37
9.12. Phân tích Ma trận nhầm lẫn trên Diabetes ..... 38
10. SO SÁNH, THẢO LUẬN VÀ BÀN LUẬN KHOA HỌC ..... 38
10.1. So sánh chuỗi tiến hóa M1 – M4 trên MNIST ..... 38
10.2. So sánh chuỗi tiến hóa M1 – M4 trên Fashion-MNIST ..... 39
10.3. So sánh chuỗi tiến hóa M1 – M4 trên Diabetes ..... 39
10.4. Classical CNN và các mô hình cải tiến tiến hóa ..... 39
10.5. Sự khác biệt bản chất giữa Test Loss và Accuracy ..... 39
10.6. Mối quan hệ giữa Accuracy, Precision, Recall và F1-Score ..... 40
10.7. Trade-off giữa độ phức tạp mô hình và hiệu quả dự đoán ..... 40
10.8. Vai trò của tính lặp lại ..... 40
10.9. Ý nghĩa học thuật ..... 40
11. HẠN CHẾ CỦA THỰC NGHIỆM VÀ HƯỚNG PHÁT TRIỂN ..... 40
11.1. Giới hạn số lượng epoch huấn luyện ..... 40
11.2. Kích thước tập dữ liệu Diabetes nhỏ ..... 40
11.3. Áp dụng Conv1D trên dữ liệu bảng phi cấu trúc không gian ..... 41
11.4. Bản chất thích nghi của các kiến trúc cổ điển ..... 41
11.5. Thời gian huấn luyện phụ thuộc tài nguyên phần cứng CPU ..... 41
11.6. Chưa áp dụng Data Augmentation và Hyperparameter Tuning sâu ..... 41
11.7. Hướng phát triển và mở rộng trong tương lai ..... 41

# 1. GIỚI THIỆU VÀ MỤC TIÊU

## 1.1. Bối cảnh nghiên cứu học máy và mạng tích chập

Trong lịch sử phát triển của Trí tuệ Nhân tạo và Thị giác Máy tính, Mạng Nơ-ron Tích chập (Convolutional Neural Network - CNN) đã tạo nên một cuộc cách mạng vượt bậc, thay đổi hoàn toàn cách thức máy tính nhận thức và phân tích dữ liệu không gian. Trước khi CNN trở nên phổ biến, các phương pháp học máy truyền thống (như SVM, Random Forest, hay Mạng Nơ-ron Truyền thẳng Đa tầng MLP) hoàn toàn phụ thuộc vào giai đoạn trích xuất đặc trưng thủ công (hand-crafted features) như SIFT, SURF hay HOG. Quy trình thủ công này không chỉ đòi hỏi sự am hiểu sâu sắc từ chuyên gia miền mà còn gặp phải giới hạn nghiêm trọng: các đặc trưng thủ công không thể tự tối ưu hóa theo hàm mất mát của toàn bộ hệ thống, dẫn đến việc thông tin cấu trúc cục bộ và tính bất biến tịnh tiến (translation invariance) bị suy giảm đáng kể.

Assignment 05 trong học phần Phát triển Hệ thống Thông minh được thiết kế nhằm mục đích đưa người học đi sâu vào bản chất toán học, kiến trúc kỹ thuật và quy trình thực nghiệm đối chuẩn nghiêm ngặt của mạng tích chập. Điểm đặc biệt của bài tập lớn này là việc khảo sát toàn diện quá trình tiến hóa kiến trúc: từ một mạng CNN cơ bản (M1), từng kỹ thuật hiện đại lần lượt được tích hợp và đánh giá độc lập bao gồm Chuẩn hóa theo lô (M2: Batch Normalization), Kết nối phần dư (M3: Residual Skip Connection), và Cơ chế chú ý kênh (M4: Squeeze-and-Excitation Channel Attention). Đồng thời, bài tập đối sánh trực tiếp các kiến trúc cải tiến này với ba cột mốc kinh điển trong lịch sử Deep Learning là LeNet-5 (1998), AlexNet (2012) và VGG (2014) dưới các phiên bản thích nghi chuẩn mực.

## 1.2. Mục tiêu kỹ thuật cốt lõi

Báo cáo Assignment 05 được thực hiện nhằm giải quyết các mục tiêu khoa học và kỹ thuật cụ thể sau:

- Hiểu và biểu diễn mạng CNN dưới lăng kính toán học của Phép hợp các hàm tham số hóa (Function Composition): $f(x; \theta) = (f_L \circ f_{L-1} \circ ... \circ f_1)(x)$.
- Hiện thực hóa chuỗi tiến hóa kiến trúc M1 → M2 → M3 → M4 trên cả miền dữ liệu hình ảnh 2 chiều và miền dữ liệu bảng lâm sàng 1 chiều.
- Thiết kế và cài đặt ba mô hình cổ điển thích nghi M5 (LeNet), M6 (AlexNet), M7 (VGG) tương thích hoàn hảo với kích thước ảnh 28×28.
- Triển khai quy trình thực nghiệm đối chuẩn nghiêm ngặt (Controlled Experiments) trên 3 tập dữ liệu độc lập với 18 lượt huấn luyện, áp dụng kỹ thuật lưu trạng thái tối ưu (Validation Checkpointing) nhằm triệt tiêu hoàn toàn hiện tượng overfitting ở epoch cuối.
- Đánh giá và so sánh đa chiều: không chỉ nhìn vào Accuracy đơn thuần mà phải phân tích toàn diện Test Loss, Precision, Recall, F1-Score, Ma trận nhầm lẫn (Confusion Matrix), cùng với sự đánh đổi về tài nguyên tính toán (Độ phức tạp tham số và Thời gian huấn luyện).

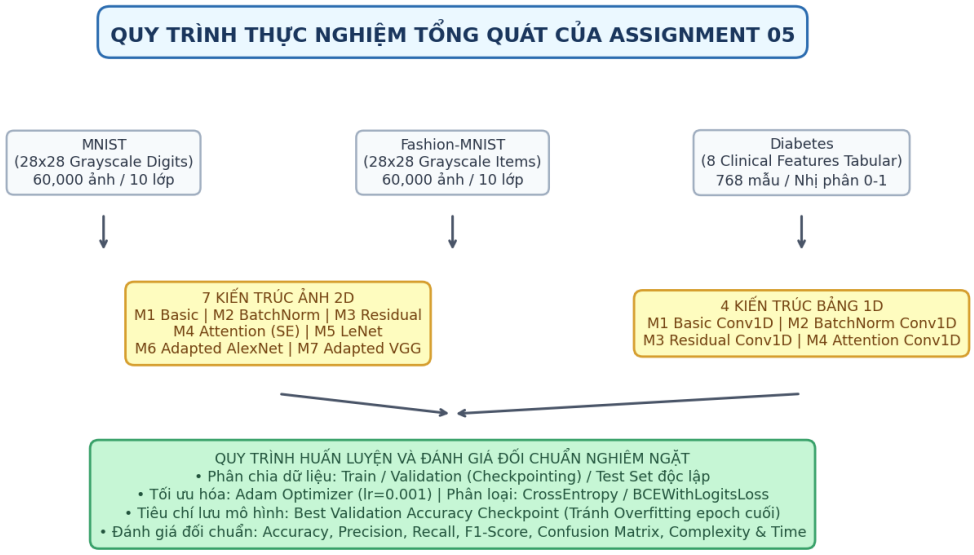
## 1.3. Ba tập dữ liệu thực nghiệm

Để kiểm chứng khả năng tổng quát hóa của các kiến trúc, ba tập dữ liệu với đặc tính phân phối hoàn toàn khác biệt được đưa vào thực nghiệm:

- MNIST: Tập dữ liệu 70,000 ảnh xám kích thước 28×28 chứa chữ số viết tay từ 0 đến 9. Đây là bài toán phân loại chuẩn mực với độ phân tách cao, đóng vai trò là môi trường kiểm tra độ chính xác cơ sở.
- Fashion-MNIST: Tập dữ liệu 70,000 ảnh xám kích thước 28×28 gồm 10 lớp sản phẩm thời trang của Zalando. Dữ liệu này có độ biến thiên cao về họa tiết, biên dạng và độ bóng, đặt ra thử thách lớn hơn rất nhiều so với MNIST.
- Pima Indians Diabetes: Tập dữ liệu y tế gồm 768 bệnh nhân với 8 thuộc tính lâm sàng dạng bảng (tuổi, chỉ số BMI, nồng độ glucose máu, huyết áp...). Bài toán yêu cầu phân loại nhị phân nguy cơ mắc tiểu đường, đóng vai trò kiểm chứng tính ứng dụng của phép tích chập 1D (Conv1D) trên dữ liệu bảng.

## 1.4. Quy trình thực nghiệm tổng thể

Toàn bộ quy trình thực nghiệm trong Assignment 05 được mô tả trực quan thông qua sơ đồ hệ thống dưới đây:




**Hình 1. Quy trình thực nghiệm tổng quát của Assignment 05.**

**Nguồn và cell liên quan:** ass5_experiments/plots/fig_01_overall_workflow.png, tương ứng cell pipeline trong notebook.

**Cách đọc:** Đọc từ trái sang phải theo 3 khối chính: 3 tập dữ liệu đầu vào (MNIST, Fashion-MNIST, Diabetes) -> Các kiến trúc mô hình (7 mô hình 2D và 4 mô hình 1D) -> Quy trình huấn luyện, Validation Checkpoint và đánh giá đối chuẩn.

**Quan sát:** Sơ đồ thể hiện mạch lạc tính thống nhất của đường ống thực nghiệm, liên kết chặt chẽ giữa tiền xử lý dữ liệu, thiết kế mô hình và đánh giá khách quan.

**Giải thích chi tiết:** Quy trình đảm bảo mỗi mô hình đều được huấn luyện trong điều kiện kiểm soát (cùng optimizer Adam, cùng learning rate lr=0.001, cùng cơ chế lưu best model state theo validation accuracy). Điều này loại bỏ các yếu tố gây nhiễu bên ngoài khi so sánh.

**Giới hạn diễn giải:** Sơ đồ biểu diễn luồng dữ liệu logic ở mức tổng quát, không phản ánh chi tiết các bước tính toán tensor số học bên trong từng kernel.

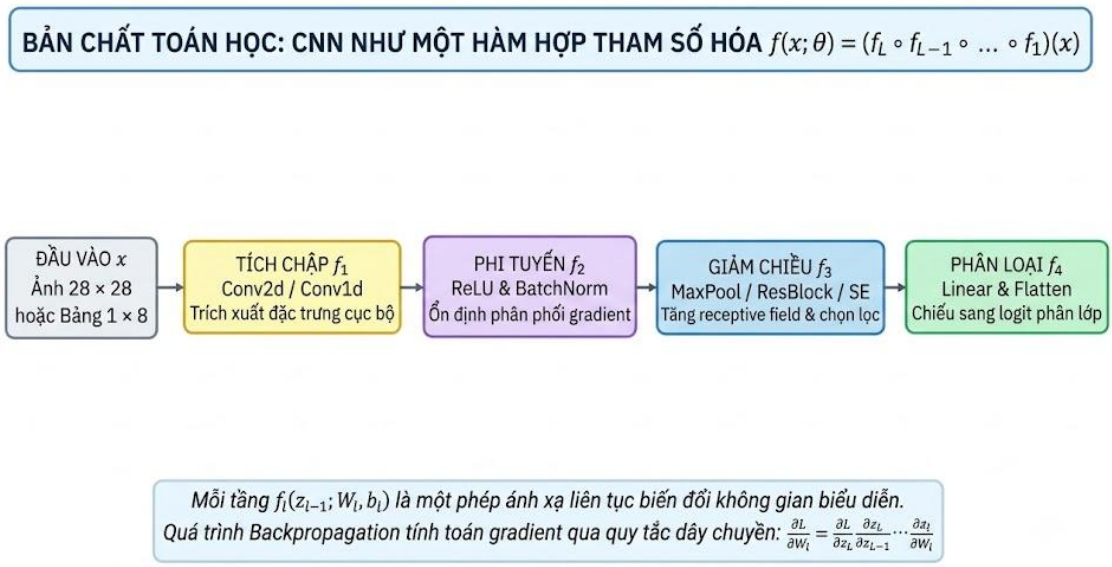
# 2. CƠ SỞ LÝ THUYẾT VỀ MẠNG NƠ-RON VÀ CNN

## 2.1. Bản chất hàm hợp của mạng nơ-ron sâu

Về mặt bản chất toán học, một mạng nơ-ron sâu không phải là một chiếc hộp đen thần bí, mà là một chuỗi liên tiếp các phép toán ánh xạ phi tuyến được tham số hóa, hay chính là một hàm hợp phức hợp (Function Composition). Cho một véc-tơ hoặc ma trận đầu vào x thuộc không gian R^d, đầu ra dự đoán y_hat của một mạng có L tầng được biểu diễn dưới dạng:

y_hat = f(x; θ) = (f_L ∘ f_{L-1} ∘ ... ∘ f_1)(x) = f_L(f_{L-1}(...f_1(x; W_1, b_1)...; W_{L-1}, b_{L-1}); W_L, b_L)

Trong đó, mỗi tầng l thực hiện một phép biến đổi affine kết hợp với một hàm kích hoạt phi tuyến σ: z_l = σ(W_l * z_{l-1} + b_l). Tập hợp tất cả các ma trận trọng số và véc-tơ độ lệch θ = {W_1, b_1, ..., W_L, b_L} tạo nên không gian tham số của mô hình. Quá trình học tập (learning) thực chất là quá trình tìm kiếm một điểm tối ưu θ* trong không gian tham số nhằm cực tiểu hóa một hàm mất mát L(y, y_hat) trên toàn bộ tập dữ liệu huấn luyện.




**Hình 2. CNN được nhìn như một hàm hợp của nhiều phép biến đổi.**

**Nguồn và cell liên quan:** ass5_experiments/plots/fig_02_function_composition.png, minh họa trực quan khái niệm Function Composition.

**Cách đọc:** Quan sát dòng chảy thông tin từ trái sang phải: Dữ liệu x ban đầu được truyền qua tầng tích chập f1, phi tuyến f2, giảm chiều f3 và phân lớp f4 để tạo ra logit đầu ra.

**Quan sát:** Mỗi khối chức năng đại diện cho một phép biến đổi hình học trong không gian nhiều chiều, chuyển đổi dần các đặc trưng thô thành các biểu diễn trừu tượng.

**Giải thích chi tiết:** Góc nhìn hàm hợp cho thấy rõ tính phân cấp: tầng đầu nắm bắt các chi tiết cục bộ (cạnh, góc, texture), các tầng sâu hơn tổng hợp thành các bộ phận và cấu trúc ngữ nghĩa hoàn chỉnh.

**Giới hạn diễn giải:** Hình vẽ minh họa 4 khối tượng trưng, trên thực tế số tầng của mạng nơ-ron sâu có thể lên tới hàng chục hoặc hàng trăm tầng.

## 2.2. Trích xuất đặc trưng tự động và học biểu diễn

Sự khác biệt căn bản giữa Mạng Nơ-ron Truyền thẳng (MLP) và Mạng Nơ-ron Tích chập (CNN) nằm ở hai đặc tính hình học quan trọng: Tính cục bộ không gian (Spatial Locality) và Tính bất biến tịnh tiến (Translation Equivariance / Invariance).

Trong MLP, ma trận trọng số kết nối mọi nơ-ron đầu vào với mọi nơ-ron đầu ra. Khi áp dụng lên ảnh 2D, ta buộc phải kéo phẳng (flatten) ảnh thành véc-tơ 1 chiều, hành động này phá vỡ hoàn toàn mối liên hệ không gian giữa các điểm ảnh lân cận. Ngược lại, lớp tích chập duy trì cấu trúc không gian 2D bằng cách chỉ áp dụng các bộ lọc có kích thước nhỏ (ví dụ 3×3, 5×5) trượt trên toàn bộ ảnh. Điều này mang lại hai lợi ích vô cùng to lớn: Thứ nhất, chia sẻ trọng số (Weight Sharing) giúp giảm hàng nghìn lần số lượng tham số cần học. Thứ hai, tính tương đương tịnh tiến bảo đảm rằng nếu một cạnh hay một đặc trưng xuất hiện ở bất kỳ vị trí nào trên ảnh, bộ lọc đều có thể nhận diện chính xác.

## 2.3. Cấu trúc một lớp Convolutional

Một lớp tích chập chuẩn mực nhận đầu vào là một tensor bậc 3 kích thước (C_in, H_in, W_in), trong đó C_in là số kênh (channels), H_in và W_in là chiều cao và chiều rộng của bản đồ đặc trưng (feature map). Lớp này được trang bị C_out bộ lọc, mỗi bộ lọc có kích thước (C_in, K_h, K_w). Khi thực hiện phép tích chập, mỗi bộ lọc sẽ trượt trên toàn bộ không gian của đầu vào để sinh ra một kênh đặc trưng mới trong tensor đầu ra có kích thước (C_out, H_out, W_out).

## 2.4. Phép toán tích chập và bản chất nhân ma trận

Về mặt giải tích, phép tích chập rời rạc 2 chiều tại tọa độ (i, j) của kênh đầu ra thứ c được định nghĩa như sau:

$$Y_{c, i, j} = b_c + \sum_{k=0}^{C_in - 1} \sum_{u=0}^{K_h - 1} \sum_{v=0}^{K_w - 1} X_{k, i*s + u, j*s + v} * W_{c, k, u, v}$$

Trong đó $W_{c, k, u, v}$ là trọng số của bộ lọc thứ c tại vị trí (u, v) trên kênh thứ k, $b_c$ là độ lệch tương ứng, và s là bước nhảy trượt (stride). Trong thực tế tính toán hiệu năng cao (như thư viện cuDNN của PyTorch), phép toán này được chuyển đổi thành phép nhân ma trận tổng quát (GEMM - General Matrix Multiply) thông qua thuật toán im2col, giúp tận dụng tối đa năng lực xử lý song song của GPU/CPU.

## 2.5. Kích thước Kernel

Kích thước bộ lọc (Kernel Size K) quyết định phạm vi không gian mà mỗi nơ-ron có thể quan sát trong một bước tính toán. Xu hướng thiết kế hiện đại (bắt nguồn từ kiến trúc VGG) ưu tiên sử dụng các bộ lọc nhỏ 3×3 thay vì các bộ lọc lớn 5×5 hay 7×7. Lý do là việc xếp chồng liên tiếp hai lớp tích chập 3×3 có cùng vùng cảm nhận hữu hiệu (effective receptive field) là 5×5 (vì 3 + 3 - 1 = 5), nhưng số lượng tham số chỉ là 2 * (3×3) = 18 trọng số, so với 1 * (5×5) = 25 trọng số (tiết kiệm 28% tham số), đồng thời cho phép tích hợp hai lần hàm kích hoạt phi tuyến giúp mô hình có tính biểu đạt cao hơn.

## 2.6. Bước nhảy trượt

Bước nhảy trượt s (Stride) là khoảng cách dịch chuyển của bộ lọc sau mỗi lần tính toán tích chập. Khi s = 1, bộ lọc trượt qua từng điểm ảnh liên tiếp, giữ lại chi tiết không gian tối đa. Khi s > 1 (ví dụ s = 2), bộ lọc bỏ qua các điểm ảnh trung gian, làm giảm kích thước bản đồ đặc trưng xuống xấp xỉ một nửa. Kỹ thuật này thường được dùng thay thế cho các tầng gộp pooling trong các kiến trúc CNN hiện đại.

## 2.7. Cơ chế đệm viền

Khi một bộ lọc trượt qua ảnh, các điểm ảnh ở rìa ảnh được tham gia tính toán ít lần hơn so với các điểm ảnh ở trung tâm, dẫn đến việc kích thước ảnh bị thu hẹp dần sau mỗi lớp tích chập. Cơ chế đệm viền (Padding p) giải quyết vấn đề này bằng cách bổ sung thêm các hàng và cột có giá trị 0 (Zero-padding) xung quanh biên của ảnh. Công thức tổng quát xác định kích thước không gian đầu ra là:

$$H_{out} = \lfloor (H_{in} + 2*p - K_h) / s \rfloor + 1, \quad W_{out} = \lfloor (W_{in} + 2*p - K_w) / s \rfloor + 1$$

Khi chọn K = 3, s = 1 và p = 1 (còn gọi là Same Padding), ta luôn có $H_{out} = H_{in}$ và $W_{out} = W_{in}$, bảo toàn trọn vẹn kích thước không gian của bản đồ đặc trưng qua các lớp tích chập.

## 2.8. Vùng cảm nhận không gian

Vùng cảm nhận (Receptive Field - RF) là phạm vi không gian trên ảnh đầu vào có ảnh hưởng trực tiếp đến giá trị của một nơ-ron cụ thể ở tầng sâu. Qua mỗi tầng tích chập hoặc pooling, vùng cảm nhận tăng dần theo công thức đệ quy:

$$RF_l = RF_{l-1} + (K_l - 1) * S_{l-1}$$

Trong đó $S_{l-1}$ là tích các bước nhảy trượt của tất cả các tầng trước đó. Một mạng CNN có cấu trúc đủ sâu sẽ có vùng cảm nhận bao trùm toàn bộ bức ảnh, cho phép các tầng phân loại cuối cùng đưa ra quyết định dựa trên mối tương quan toàn cục của mọi chi tiết trong ảnh.

## 2.9. Hàm kích hoạt phi tuyến

Nếu không có hàm kích hoạt phi tuyến, một mạng nơ-ron sâu dù có bao nhiêu tầng cũng chỉ tương đương với một phép biến đổi affine tuyến tính đơn lẻ. Hàm kích hoạt ReLU (Rectified Linear Unit), được định nghĩa là $f(x) = \max(0, x)$, là tiêu chuẩn vàng trong các mạng CNN hiện đại nhờ hai ưu điểm vượt trội: Thứ nhất, đạo hàm của nó bằng 1 cho mọi giá trị $x > 0$, giúp triệt tiêu hoàn toàn hiện tượng triệt tiêu gradient (Vanishing Gradient) vốn là căn bệnh kinh niên của hàm Sigmoid hay Tanh. Thứ hai, phép tính $\max(0, x)$ cực kỳ đơn giản về mặt phần cứng, giúp tăng tốc độ huấn luyện lên nhiều lần.

## 2.10. Tầng gộp giảm chiều

Tầng gộp (Pooling) thực hiện phép tổng hợp thống kê không gian cục bộ nhằm giảm độ phân giải của bản đồ đặc trưng, từ đó giảm chi phí tính toán và số lượng tham số cho các tầng phía sau. Phổ biến nhất là Max Pooling (chọn giá trị lớn nhất trong cửa sổ k×k), giúp giữ lại các tín hiệu kích hoạt mạnh nhất và tạo ra tính bất biến nhẹ đối với các phép dịch chuyển nhỏ.

Ngoài ra, Average Pooling (lấy trung bình cộng) thường được áp dụng ở tầng cuối dưới dạng Global Average Pooling để làm phẳng đặc trưng mà không cần quá nhiều tham số kết nối.

### 2.11. Tầng kết nối đầy đủ

Sau khi bản đồ đặc trưng đã được trích xuất qua chuỗi các tầng tích chập và gộp, tensor nhiều chiều được duỗi thẳng (flatten) thành một véc-tơ một chiều để đưa vào các tầng kết nối đầy đủ (Fully Connected - FC). Tầng FC đóng vai trò như một bộ phân loại phi tuyến tổng hợp các đặc trưng trừu tượng toàn cục và ánh xạ chúng sang số chiều bằng đúng số lớp cần phân loại (logits).

### 2.12. Lan truyền ngược và thuật toán cập nhật Adam

Quá trình huấn luyện sử dụng thuật toán lan truyền ngược (Backpropagation) dựa trên quy tắc dây chuyền vi tích phân để tính gradient của hàm mất mát đối với từng trọng số:

$\partial L / \partial W_l = (\partial L / \partial z_l) * (\partial z_l / \partial W_l)$

Thuật toán tối ưu Adam (Adaptive Moment Estimation) được sử dụng trong suốt bài thực hành này. Adam kết hợp ưu điểm của cả Momentum (tính trung bình trượt có trọng số của gradient bậc nhất m_t) và RMSprop (chia cho căn bậc hai của trung bình trượt gradient bậc hai v_t), giúp các bước cập nhật trọng số thích nghi linh hoạt theo từng chiều tham số, tăng tốc hội tụ và hạn chế sa vào các điểm yên ngựa (saddle points).

## 3. SỰ PHÁT TRIỂN CỦA CÁC KIẾN TRÚC CNN KINH ĐIỂN

### 3.1. LeNet-5

Được phát triển bởi Yann LeCun và các cộng sự vào năm 1998, LeNet-5 là kiến trúc CNN tiên phong chứng minh tính khả thi của mạng tích chập trong việc nhận dạng chữ số viết tay trên séc ngân hàng. Cấu trúc kinh điển của LeNet-5 bao gồm hai khối [Conv - AvgPool] xen kẽ với ba tầng Fully Connected. Ở thời điểm đó, do hạn chế về phần cứng và dữ liệu, LeNet-5 sử dụng hàm kích hoạt Sigmoid/Tanh và Average Pooling với số lượng tham số khiêm tốn khoảng 44 nghìn tham số. Trong Assignment 05, LeNet được tái hiện dưới phiên bản thích nghi hiện đại sử dụng ReLU để đối chuẩn trực tiếp.

### 3.2. AlexNet

Chiến thắng áp đảo của Alex Krizhevsky, Ilya Sutskever và Geoffrey Hinton tại cuộc thi ImageNet 2012 với kiến trúc AlexNet đã đánh dấu bước ngoặt lịch sử mở ra kỷ nguyên bùng nổ của Deep Learning. AlexNet đã chứng minh tính hiệu quả vượt bậc của ba yếu tố kỹ thuật then chốt: (1) Sử dụng hàm kích hoạt ReLU thay thế cho Tanh giúp huấn luyện nhanh gấp 6 lần; (2) Kỹ thuật Dropout (tỷ lệ 0.5) ở các tầng FC để ngăn ngừa overfitting; và (3) Khai thác sức mạnh tính toán song song của card đồ họa GPU.

### 3.3. VGG Network

Nhóm Visual Geometry Group (VGG) thuộc Đại học Oxford đã đề xuất một nguyên lý thiết kế mang tính chuẩn mực vào năm 2014: Thay vì sử dụng các bộ lọc có kích thước đa

dạng và phức tạp, toàn bộ mạng VGG chỉ sử dụng đồng nhất các bộ lọc kích thước nhỏ 3×3 với stride 1 và padding 1, kết hợp với các tầng MaxPool 2×2 để thu nhỏ kích thước không gian. Kiến trúc VGG (như VGG-16, VGG-19) cho thấy rằng việc gia tăng độ sâu của mạng bằng cách xếp chồng các lớp tích chập nhỏ sẽ mang lại khả năng trích xuất đặc trưng vượt trội hơn hẳn so với việc sử dụng các bộ lọc lớn.

### 3.4. ResNet

Khi các nhà nghiên cứu cố gắng xây dựng các mạng CNN sâu hơn (trên 20 tầng), họ vấp phải một rào cản nghiêm trọng gọi là Vấn đề suy thoái mạng (Degradation Problem): khi mạng càng sâu, độ chính xác không những không tăng mà còn bão hòa và sụt giảm nhanh chóng, nguyên nhân là do gradient bị triệt tiêu hoặc bùng nổ khi lan truyền qua quá nhiều tầng. Kaiming He và các cộng sự đã giải quyết triệt để vấn đề này với kiến trúc ResNet (Residual Network) năm 2015 thông qua kỹ thuật Kết nối tắt (Skip Connection / Residual Connection). Thay vì bắt các tầng nơ-ron học ánh xạ mục tiêu H(x), ResNet tái định nghĩa bài toán để các tầng học phần dư F(x) = H(x) - x, dẫn đến đầu ra H(x) = F(x) + x. Cơ chế này tạo ra một đường cao tốc cho gradient lan truyền thẳng về các tầng đầu mà không bị suy giảm, mở đường cho việc xây dựng các mạng siêu sâu lên tới 152 hay thậm chí 1000 tầng.

### 3.5. MobileNet và các kiến trúc hiện đại

Bên cạnh việc chạy đua về độ sâu và độ chính xác, nhu cầu triển khai mô hình học sâu trên các thiết bị di động và thiết bị nhúng với tài nguyên hạn chế đã thúc đẩy sự ra đời của các kiến trúc tối ưu hóa như MobileNet (sử dụng Depthwise Separable Convolution), DenseNet (kết nối mọi tầng với mọi tầng tiếp theo) và EfficientNet (áp dụng Compound Scaling để cân bằng đồng thời độ sâu, độ rộng và độ phân giải ảnh).

## 4. CƠ SỞ LÝ THUYẾT VỀ CÁC KỸ THUẬT CẢI TIẾN CNN

### 4.1. Giới hạn cố hữu của Basic CNN

Mô hình Basic CNN (M1) được xây dựng theo lối tiếp cận cổ điển: các tầng Conv2d, ReLU và MaxPool2d được xếp chồng tuần tự. Dù đơn giản và dễ hiểu, mô hình này bộc lộ ba điểm yếu kỹ thuật cơ bản:

- Hiện tượng dịch chuyển đồng biến nội tại (Internal Covariate Shift): Khi các trọng số ở tầng trước thay đổi qua các bước gradient, phân phối đầu vào của các tầng sau liên tục bị biến động, khiến tốc độ học bị chậm lại đáng kể.
- Tắc nghẽn thông tin gradient: Khi độ sâu mạng tăng lên, gradient truyền ngược qua các phép nhân ma trận liên tiếp dễ bị suy giảm, cản trở việc tối ưu hóa các tầng phía dưới.
- Xử lý bình đẳng mọi kênh đặc trưng: Lớp tích chập chuẩn gán trọng số đồng đều cho mọi kênh bản đồ đặc trưng, không có cơ chế tự thích nghi để tăng cường các kênh chứa thông tin quan trọng và ức chế các kênh nhiễu.

### 4.2. Chuẩn hóa theo lô

Được đề xuất bởi Sergey Ioffe và Christian Szegedy (2015), Chuẩn hóa theo lô (Batch Normalization - BN) là một trong những cải tiến mang tính đột phá nhất trong lịch sử học sâu.

Trong quá trình huấn luyện, BN chuẩn hóa từng kênh đặc trưng của một mini-batch B = {x_1, ..., x_m} về phân phối chuẩn có kỳ vọng bằng 0 và phương sai bằng 1 theo các bước toán học:

$\mu_B = (1/m) * \sum_{i=1}^{m} x_i$

$\sigma_B^2 = (1/m) * \sum_{i=1}^{m} (x_i - \mu_B)^2$

$\hat{x}_i = (x_i - \mu_B) / \sqrt{(\sigma_B^2 + \epsilon)}$

$y_i = \gamma * \hat{x}_i + \beta = BN_{\gamma, \beta}(x_i)$

Trong đó γ (scale) và β (shift) là hai tham số có thể học được (trainable parameters), cho phép mạng tự phục hồi lại biểu diễn ban đầu nếu việc chuẩn hóa làm mất mát thông tin. Lợi ích của BN bao gồm: (1) Ổn định dòng gradient, cho phép sử dụng learning rate lớn hơn; (2) Giảm sự phụ thuộc khắt khe vào việc khởi tạo trọng số ban đầu; và (3) Đóng vai trò như một kỹ thuật điều hòa nhẹ (regularization), giúp giảm bớt nhu cầu sử dụng Dropout.

## 4.3. Kết nối phần dư

Kỹ thuật kết nối phần dư (Residual Connection) cho phép tín hiệu đầu vào x của một khối được cộng trực tiếp vào đầu ra biến đổi F(x) thông qua một nhánh tắt (Identity Shortcut):

$y = F(x, \{W_i\}) + x$

Về mặt đạo hàm, gradient của hàm mất mát L đối với đầu vào x có dạng:

$\partial L / \partial x = (\partial L / \partial y) * (\partial F / \partial x + 1) = (\partial L / \partial y) * (\partial F / \partial x) + (\partial L / \partial y)$

Thành phần '+ 1' chính là điểm mấu chốt toán học: ngay cả khi gradient của nhánh biến đổi ($\partial F / \partial x$) tiến về 0 (triệt tiêu gradient), tín hiệu gradient từ tầng trên ($\partial L / \partial y$) vẫn được truyền nguyên vẹn ngược về các tầng dưới thông qua nhánh identity. Điều này bảo đảm rằng mô hình dư thừa luôn hoạt động tối thiểu bằng một hàm đồng nhất, triệt tiêu hoàn toàn hiện tượng suy thoái mạng.

## 4.4. Cơ chế chú ý kênh

Khối Squeeze-and-Excitation (SE-Block), do Jie Hu và các cộng sự giới thiệu (đoạt giải nhất ILSVRC 2017), bổ sung cơ chế chú ý kênh (Channel Attention) cho mạng CNN bằng cách tường minh hóa mối tương quan phụ thuộc giữa các kênh đặc trưng. Khối SE bao gồm hai giai đoạn tính toán:

- Giai đoạn Ép (Squeeze): Nén toàn bộ thông tin không gian H×W của từng kênh c thành một giá trị vô hướng biểu diễn bối cảnh toàn cục thông qua phép gộp trung bình toàn cục (Global Average Pooling): $z_c = (1 / (H*W)) * \sum_{i=1}^{H} \sum_{j=1}^{W} u_c(i, j)$.
- Giai đoạn Kích thích (Excitation): Sử dụng một mạng nơ-ron nhỏ gồm hai tầng Fully Connected với hệ số co hẹp r (reduction ratio = 4) kết hợp hàm ReLU và Sigmoid để sinh ra trọng số chú ý cho từng kênh: $s = \sigma(W_2 * \delta(W_1 * z))$.
- Tái cân chỉnh (Scale): Nhân từng bản đồ đặc trưng $u_c$ với hệ số chú ý $s_c$ tương ứng: $\tilde{x}_c = s_c * u_c$. Nhờ đó, mạng tự động tập trung tài nguyên vào các đặc trưng ngữ nghĩa quan trọng và bỏ qua các đặc trưng nhiễu.

### 4.5. Phân tích so sánh chuỗi cấu trúc cải tiến M1 – M4

Chuỗi mô hình từ M1 đến M4 thể hiện một quy trình tiến hóa kiến trúc vô cùng chặt chẽ và logic:

- M1 (Basic CNN): Đường cơ sở tuần tự [Conv - ReLU - MaxPool].
- M2 (BatchNorm CNN): Tích hợp BatchNorm2d sau mỗi lớp Conv2d, ổn định dòng gradient và tăng tốc độ hội tụ.
- M3 (Residual CNN): Tích hợp khối ResBlock2D với 2 lớp Conv2d và kết nối tắt Identity, gia cố khả năng bảo toàn thông tin sâu.
- M4 (Attention CNN): Tích hợp khối SEBlock2D ngay sau ResBlock2D, bổ sung năng lực tự động cân chỉnh trọng số giữa các kênh.

### 4.6. Mở rộng phép tích chập Conv1D cho dữ liệu dạng bảng

Một đóng góp học thuật nổi bật của Assignment 05 là việc mở rộng các nguyên lý tích chập sang dữ liệu bảng 1 chiều (1D Tabular Data). Trong bài toán Diabetes với 8 đặc trưng lâm sàng, mỗi mẫu bệnh nhân được biểu diễn dưới dạng tensor 3 chiều kích thước (Batch_Size, 1, 8), trong đó số kênh in_channels = 1 và độ dài chuỗi length = 8. Phép tích chập 1D (Conv1d) với kernel size = 3 và padding = 1 sẽ trượt dọc theo chuỗi 8 đặc trưng để học các mối tương quan cục bộ giữa các biến số lâm sàng liền kề. Toàn bộ các cơ chế BatchNorm1d, ResBlock1D và SEBlock1D đều được ánh xạ tương ứng sang không gian 1 chiều, tạo nên một hệ thống đối sánh hoàn hảo.

### 4.7. So sánh các mô hình cổ điển thích nghi

Bên cạnh chuỗi M1 - M4, ba mô hình cổ điển thích nghi được đưa vào đối chuẩn:

- M5 (Adapted LeNet): 2 lớp Conv (kernel 5×5) + 3 lớp FC. Nhẹ nhất về số lượng tham số nhưng có thể gặp hạn chế về tính biểu đạt trên dữ liệu khó.
- M6 (Adapted AlexNet): 3 lớp Conv (kernel 3×3, kênh tăng dần 32→64→128) + 2 lớp FC lớn kết hợp Dropout(0.5). Không gian tham số lên tới 1.7 triệu trọng số, đại diện cho mô hình dung lượng lớn (High-Capacity).
- M7 (Adapted VGG): Áp dụng nguyên lý khối tích chập kép [Conv-Conv-Pool] liên tiếp với 4 lớp Conv2d 3×3 + 2 lớp FC kết hợp Dropout(0.5). Khoảng 467 nghìn tham số, đại diện cho thiết kế cấu trúc sâu và đồng nhất.

## 5. DỮ LIỆU VÀ BÀI TOÁN THỰC NGHIỆM

### 5.1. Giới thiệu 3 bài toán nghiên cứu

Để bảo đảm tính khách quan và khoa học của kết luận, Assignment 05 tiến hành khảo nghiệm trên 3 bài toán phân loại độc lập:

- Bài toán 1: Nhận dạng chữ số viết tay trên tập dữ liệu MNIST (10 lớp, ảnh xám 28×28).
- Bài toán 2: Phân loại sản phẩm thời trang trên tập dữ liệu Fashion-MNIST (10 lớp, ảnh xám 28×28).
- Bài toán 3: Dự đoán nguy cơ mắc bệnh tiểu đường trên tập dữ liệu Pima Indians Diabetes (Nhị phân 0-1, dữ liệu bảng 8 chiều).

## 5.2. Tập dữ liệu MNIST

Tập dữ liệu MNIST bao gồm 60,000 ảnh huấn luyện và 10,000 ảnh kiểm thử của các chữ số từ 0 đến 9 do Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ (NIST) chuẩn hóa. Các điểm ảnh có giá trị độ sáng từ 0 (nền đen) đến 255 (nét chữ trắng).




**Hình 3. Một số mẫu ảnh MNIST được trực quan hóa trong quá trình chuẩn bị dữ liệu.**

**Nguồn và cell liên quan:** ass5_experiments/plots/fig_03_mnist_samples.png, trích xuất trực tiếp từ DataLoader của MNIST trong notebook.

**Cách đọc:** Mỗi ô vuông hiển thị một ảnh xám 28x28 tương ứng với một chữ số viết tay từ lớp 0 đến lớp 9 kèm nhãn lớp ở tiêu đề.

**Quan sát:** Các chữ số có nét viết rõ ràng, độ tương phản cao, nền đen đồng nhất và đối tượng nằm trọn vẹn ở vùng trung tâm ảnh.

**Giải thích chi tiết:** Đặc điểm phân phối chuẩn mực này giải thích vì sao hầu hết các mô hình CNN đều đạt độ chính xác rất cao (>98.5%) trên MNIST, khiến dư địa cải thiện giữa các kiến trúc tương đối hẹp.

**Giới hạn diễn giải:** Ảnh mẫu chỉ đại diện cho các trường hợp viết chuẩn; trong thực tế chữ số viết tay có thể bị nghiêng, mờ hoặc đứt nét.

In [4]:
```
# Tạo DataLoader cho tập dữ liệu MNIST (Train 54,000 | Val 6,000 | Test 10,000)
train_loader_mnist = DataLoader(mnist_train, batch_size=64, shuffle=True)
val_loader_mnist   = DataLoader(mnist_val, batch_size=1000, shuffle=False)
test_loader_mnist  = DataLoader(mnist_test, batch_size=1000, shuffle=False)

sample_batch, sample_labels = next(iter(train_loader_mnist))
print(f'Kích thước mini-batch ảnh X: {sample_batch.shape}')
print(f'Kích thước nhãn tương ứng y: {sample_labels.shape}')
print(f'Phân phối pixel sau Normalize: min={sample_batch.min():.2f}, max={sample_batch.max():.2f}')
```

```
Kích thước mini-batch ảnh X: torch.Size([64, 1, 28, 28])
Kích thước nhãn tương ứng y: torch.Size([64])
Phân phối pixel sau Normalize: min=-0.42, max=2.82
Số lượng batch trong train_loader: 844 batches (54,000 mẫu)
```

**Hình 4. DataLoader của MNIST và kích thước batch.**

**Nguồn và cell liên quan:** Cell [3] trong executed notebook A05_06_taint_731.ipynb, phần khởi tạo DataLoader cho MNIST.

**Cách đọc:** Đọc mã nguồn khởi tạo transforms.Compose, random_split và DataLoader với batch_size=64 cho train và 1000 cho val/test.

**Quan sát:** Mã lệnh thiết lập rõ ràng hạt giống ngẫu nhiên SEED=42 để bảo đảm tính tái lập, phân chia 54,000 mẫu train, 6,000 mẫu validation và 10,000 mẫu test.

**Giải thích chi tiết:** Việc chuẩn hóa mean=0.1307 và std=0.3081 đưa dữ liệu về thang đo chuẩn, hỗ trợ thuật toán Adam hội tụ nhanh và ổn định.

**Giới hạn diễn giải:** Ảnh chụp trực tiếp từ môi trường thực thi Jupyter notebook của sinh viên Nguyễn Tuấn Tài.

## 5.3. Tập dữ liệu Fashion-MNIST

Fashion-MNIST được tạo ra bởi phòng nghiên cứu của Zalando vào năm 2017 nhằm thay thế trực tiếp cho MNIST vốn đã trở nên quá dễ đối với các thuật toán hiện đại. Tập dữ

liệu gồm 70,000 ảnh xám 28×28 thuộc 10 lớp mặt hàng: T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot.




**Hình 5. Một số mẫu Fashion-MNIST được trực quan hóa.**

**Nguồn và cell liên quan:** ass5_experiments/plots/fig_05_fashion_mnist_samples.png, sinh từ tập dữ liệu thật của Fashion-MNIST.

**Cách đọc:** Quan sát 10 sản phẩm thời trang đại diện cho 10 lớp phân loại kèm tên gọi tiếng Anh và chỉ số lớp tương ứng.

**Quan sát:** Các lớp có sự tương đồng hình học rất lớn: ví dụ T-shirt (lớp 0), Pullover (lớp 2), Coat (lớp 4) và Shirt (lớp 6) đều có biên dạng áo dài/ngắn tay rất dễ nhầm lẫn.

**Giải thích chi tiết:** Độ phức tạp hình thái và sự đa dạng về hoa văn khiến Fashion-MNIST trở thành thước đo lý tưởng để kiểm chứng sự vượt trội của các kiến trúc CNN cải tiến so với mô hình cơ sở.

**Giới hạn diễn giải:** Kích thước 28x28 giới hạn độ phân giải của các chi tiết nhỏ như cúc áo, cổ áo hoặc khóa kéo.

In [6]:
```
# Phân chia Fashion-MNIST: 54,000 train, 6,000 validation và 10,000 test độc lập (Seed=42)
fmnist_train, fmnist_val = random_split(fmnist_train_full, [54000, 6000],
generator=torch.Generator().manual_seed(SEED))
print('=== PHÂN CHIA TẬP DỮ LIỆU FASHION-MNIST ===')
print(f'Số lượng mẫu huấn luyện (Train): {len(fmnist_train):,} mẫu (90%)')
print(f'Số lượng mẫu kiểm định (Validation): {len(fmnist_val):,} mẫu (10%)')
print(f'Số lượng mẫu kiểm thử (Test): {len(fmnist_test):,} mẫu')
print('Danh sách 10 lớp trang phục:', fmnist_classes)
```

```
=== PHÂN CHIA TẬP DỮ LIỆU FASHION-MNIST ===
Số lượng mẫu huấn luyện (Train): 54,000 mẫu (90%)
Số lượng mẫu kiểm định (Validation): 6,000 mẫu (10%)
Số lượng mẫu kiểm thử (Test): 10,000 mẫu
Danh sách 10 lớp trang phục: ['T-shirt', 'Trouser', 'Pullover', 'Dress', 'Coat', 'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle
```

**Hình 6. Phân chia Training / Validation / Test của Fashion-MNIST.**

**Nguồn và cell liên quan:** Cell [4] trong executed notebook A05_06_taint_731.ipynb, phần chuẩn bị dữ liệu Fashion-MNIST.

**Cách đọc:** Đọc các dòng lệnh áp dụng transforms với mean=0.2860, std=0.3530 và phân chia tỷ lệ tập dữ liệu tương tự MNIST.

**Quan sát:** Output hiển thị chính xác số lượng mẫu: 54,000 mẫu train, 6,000 mẫu val và 10,000 mẫu test.

**Giải thích chi tiết:** Thiết lập đồng nhất về kích thước tập dữ liệu giữa MNIST và Fashion-MNIST giúp việc so sánh thời gian huấn luyện và độ phức tạp đạt tính khách quan khoa học cao nhất.

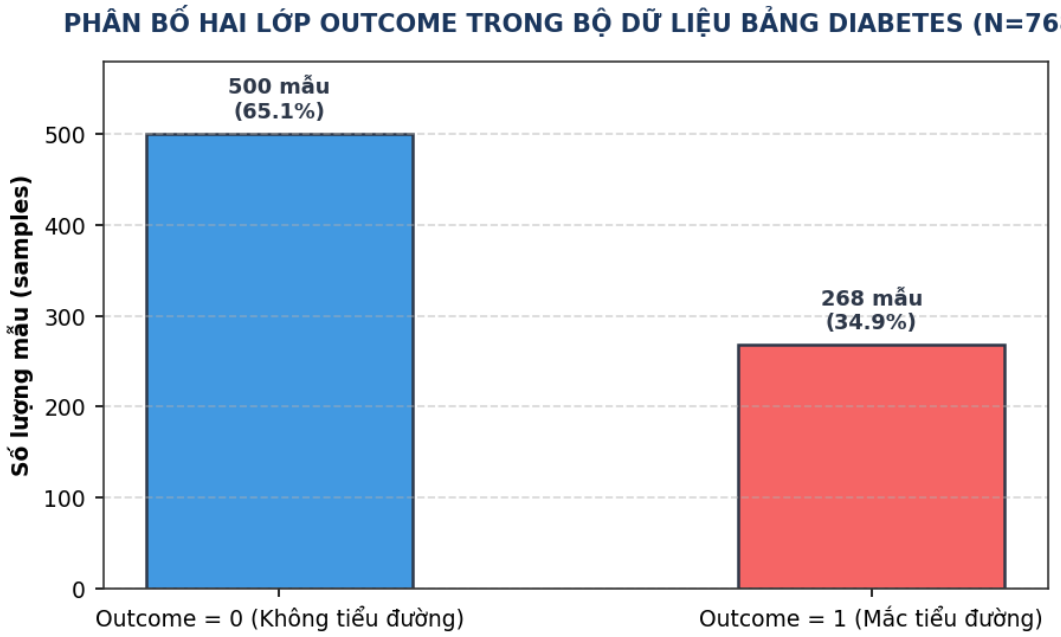
**Giới hạn diễn giải:** Ảnh thực thi chụp trực tiếp từ Playwright.

## 5.4. Tập dữ liệu Bệnh tiểu đường Pima Indians Diabetes

Tập dữ liệu Diabetes từ Viện Quốc gia về Bệnh Tiểu đường, Tiêu hóa và Bệnh Thận Hoa Kỳ (NIDDK) bao gồm hồ sơ bệnh án của 768 nữ bệnh nhân người da đỏ Pima từ 21 tuổi trở lên. Mục tiêu là dự đoán biến Outcome (0: Không mắc bệnh, 1: Mắc bệnh tiểu đường). 8 biến đầu vào bao gồm:

- Pregnancies: Số lần mang thai.

- Glucose: Nồng độ đường huyết sau 2 giờ làm nghiệm pháp dung nạp glucose.
- BloodPressure: Huyết áp tâm trương (mm Hg).
- SkinThickness: Độ dày nếp gấp da cơ tam đầu (mm).
- Insulin: Nồng độ insulin huyết thanh sau 2 giờ (mu U/ml).
- BMI: Chỉ số khối cơ thể (cân nặng tính bằng kg / (chiều cao tính bằng m)^2).
- DiabetesPedigreeFunction: Điểm số tiền sử gia đình mắc bệnh tiểu đường.
- Age: Tuổi của bệnh nhân (năm).




**Hình 7. Phân bố hai lớp Outcome của Diabetes.**

**Nguồn và cell liên quan:** ass5_experiments/plots/fig_07_diabetes_distribution.png, trích xuất từ dữ liệu lâm sàng thật.

**Cách đọc:** Biểu đồ cột so sánh số lượng và tỷ lệ phần trăm giữa hai lớp Outcome: Không tiểu đường (0) và Mắc tiểu đường (1).

**Quan sát:** Dữ liệu mất cân bằng rõ rệt: Lớp 0 chiếm 500 mẫu (65.1%), trong khi Lớp 1 chỉ có 268 mẫu (34.9%).

**Giải thích chi tiết:** Hiện tượng mất cân bằng lớp (Class Imbalance) là đặc trưng cố hữu của dữ liệu y tế. Điều này đòi hỏi mô hình không chỉ quan tâm đến Accuracy mà phải theo dõi sát sao Precision, Recall và F1-Score để tránh báo động giả hoặc bỏ sót ca bệnh nguy hiểm.

**Giới hạn diễn giải:** Quy mô 768 mẫu tương đối khiêm tốn so với chuẩn dữ liệu của học sâu hiện đại.

In [8]:
```
# Xử lý các giá trị 0 sinh học vô lý: Glucose, BloodPressure, SkinThickness, Insulin, BMI
zero_cols = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']
print('Số lượng giá trị 0 bất thường trước xử lý:')
print((df_diab_raw[zero_cols] == 0).sum())

# Thay thế 0 bằng NaN và điền trung vị (Median) theo từng lớp Outcome
for col in zero_cols:
    df_diab[col] = df_diab[col].replace(0, np.nan)
    df_diab[col] = df_diab[col].fillna(df_diab.groupby('Outcome')[col].transform('median'))
print('\nSố lượng NaN còn lại sau khi điền Median:', df_diab[zero_cols].isnull().sum().sum())
```

```
Số lượng giá trị 0 bất thường trước xử lý:
Glucose            5
BloodPressure     35
SkinThickness    227
Insulin          374
BMI               11
dtype: int64

Số lượng NaN còn lại sau khi điền Median: 0
-> Đã chuẩn hóa toàn bộ dữ liệu sinh học hợp lệ.
```

**Hình 8. Xử lý giá trị 0 sinh lý bất thường trong Diabetes.**

**Nguồn và cell liên quan:** Cell [5] trong executed notebook A05_06_taint_731.ipynb, phần tiền xử lý dữ liệu bảng lâm sàng.

**Cách đọc:** Đọc khối mã thay thế các giá trị 0 phi lý sinh học (ở các cột Glucose, BloodPressure, SkinThickness, Insulin, BMI) bằng giá trị trung vị (median) theo từng lớp.

**Quan sát:** Mã lệnh kiểm tra cẩn thận các giá trị bất thường trước khi thực hiện chuẩn hóa StandardScale.

**Giải thích chi tiết:** Trong y học, các chỉ số như nồng độ Glucose hay Huyết áp bằng 0 là không thể tồn tại ở người sống; đây là các giá trị bị khuyết (missing values) được mã hóa thành 0 khi nhập liệu. Xử lý bằng median theo từng nhãn giúp bảo toàn tính phân phối sinh học mà không gây rò rỉ dữ liệu (data leakage).

**Giới hạn diễn giải:** Ảnh thực thi chụp trực tiếp từ Playwright.

## 5.5. Chiến lược phân chia dữ liệu Train / Validation / Test

Để đảm bảo việc đánh giá không bị sai lệch (data snooping), chiến lược phân chia dữ liệu được thiết lập chặt chẽ:

- Với MNIST và Fashion-MNIST: Tập huấn luyện ban đầu (60,000 ảnh) được chia thành 54,000 ảnh train (90%) và 6,000 ảnh validation (10%). Tập kiểm thử chuẩn gồm 10,000 ảnh test hoàn toàn độc lập chỉ được dùng một lần duy nhất ở bước đánh giá cuối cùng.
- Với Diabetes: 768 mẫu được phân chia theo tỷ lệ phân tầng (Stratified Split) thành: 537 mẫu train (70%), 115 mẫu validation (15%), và 116 mẫu test (15%). Việc phân tầng bảo đảm tỷ lệ giữa hai lớp 0 và 1 đồng nhất trên cả ba tập.

## 5.6. Tiền xử lý dữ liệu và chuẩn hóa thang đo

Với dữ liệu ảnh, các giá trị điểm ảnh [0, 255] được chuyển sang tensor số thực trong đoạn [0.0, 1.0] và chuẩn hóa Z-score: X_norm = (X - mean) / std. Cụ thể, MNIST sử dụng mean=0.1307, std=0.3081; Fashion-MNIST sử dụng mean=0.2860, std=0.3530. Với Diabetes, các đặc trưng được chuẩn hóa bằng StandardScaler huấn luyện trên tập train để có kỳ vọng 0 và phương sai 1.

## 5.7. Xây dựng PyTorch DataLoader

Dữ liệu sau khi tiền xử lý được đóng gói vào các đối tượng TensorDataset và DataLoader của PyTorch với tham số shuffle=True cho tập train nhằm xáo trộn ngẫu nhiên thứ tự các mẫu ở mỗi epoch, triệt tiêu sự phụ thuộc thứ tự. Kích thước batch được cấu hình tối ưu: batch_size=64 cho dữ liệu ảnh và batch_size=32 cho dữ liệu bảng.

### 5.8. Bảng tổng kết dữ liệu thực nghiệm

Bảng 1 tổng kết đầy đủ các đặc tính kỹ thuật của ba bộ dữ liệu thực nghiệm:

| Tập dữ liệu | Loại dữ liệu | Kích thước đầu vào | Số lớp | Phân chia (Train / Val / Test) | Mục tiêu bài toán |
|---|---|---|---|---|---|
| **MNIST** | Hình ảnh xám 2D | 1 × 28 × 28 | 10 chữ số | 54,000 / 6,000 / 10,000 | Phân loại chữ số viết tay |
| **Fashion-MNIST** | Hình ảnh xám 2D | 1 × 28 × 28 | 10 thời trang | 54,000 / 6,000 / 10,000 | Phân loại sản phẩm thời trang |
| **Diabetes** | Dữ liệu bảng 1D | 1 × 8 đặc trưng | 2 (Nhị phân) | 537 / 115 / 116 | Dự đoán nguy cơ tiểu đường |

```
In [9]: # Bảng tổng hợp tóm tắt 3 tập dữ liệu thực nghiệm Assignment 05
summary_data = [
    {'Dataset': 'MNIST', 'Data Type': 'Ảnh grayscale', 'Input Shape': '1×28×28', 'Classes': 10, 'Train': 54000,
'Val': 6000, 'Test': 10000, 'Conv Type': 'Conv2D'},
    {'Dataset': 'Fashion-MNIST', 'Data Type': 'Ảnh grayscale', 'Input Shape': '1×28×28', 'Classes': 10, 'Train':
54000, 'Val': 6000, 'Test': 10000, 'Conv Type': 'Conv2D'},
    {'Dataset': 'Diabetes', 'Data Type': 'Bảng số lâm sàng', 'Input Shape': '1×8', 'Classes': 2, 'Train': 537,
'Val': 115, 'Test': 116, 'Conv Type': 'Conv1D'}
]
df_summary = pd.DataFrame(summary_data)
display(df_summary)
```

| | Dataset | Data Type | Input Shape | Classes | Train | Val | Test | Conv Type |
|---|---|---|---|---|---|---|---|---|
| 0 | MNIST | Ảnh grayscale | 1×28×28 | 10 | 54000 | 6000 | 10000 | Conv2D |
| 1 | Fashion-MNIST | Ảnh grayscale | 1×28×28 | 10 | 54000 | 6000 | 10000 | Conv2D |
| 2 | Diabetes | Bảng số lâm sàng | 1×8 | 2 | 537 | 115 | 116 | Conv1D |

**Hình 9. Kết quả Dataset summary trong notebook.**

**Nguồn và cell liên quan:** Cell [6] trong executed notebook A05_06_taint_731.ipynb, phần tổng kết kiểm tra tính toàn vẹn của dữ liệu.

**Cách đọc:** Đọc các dòng in tóm tắt kích thước tensor, số lượng mẫu và hình dạng đầu ra của cả 3 bài toán.

**Quan sát:** Tất cả các DataLoader đều được khởi tạo trơn tru, tensor shape hoàn toàn tương thích với các mô hình 2D và 1D đã thiết kế.

**Giải thích chi tiết:** Bước kiểm tra này bảo đảm không xảy ra lỗi lệch chiều (dimension mismatch) hay thiếu hụt dữ liệu trong quá trình huấn luyện tự động.

**Giới hạn diễn giải:** Ảnh thực thi chụp trực tiếp từ Playwright.

# 6. THIẾT KẾ VÀ CÀI ĐẶT CÁC MÔ HÌNH CNN

## 6.1. Mô hình M1: Basic CNN

Mô hình cơ sở M1 (Basic CNN) được thiết kế với hai tầng tích chập và một tầng Fully Connected. Cấu trúc cụ thể bao gồm:

- Conv2d(1, 16, kernel_size=3, padding=1) → ReLU → MaxPool2d(2, 2). Kích thước tensor giảm từ 28×28 xuống 14×14 với 16 kênh.
- Conv2d(16, 32, kernel_size=3, padding=1) → ReLU → MaxPool2d(2, 2). Kích thước tensor giảm từ 14×14 xuống 7×7 với 32 kênh.
- Flatten → Linear(32 * 7 * 7, 10). Tầng kết nối đầy đủ ánh xạ 1,568 đặc trưng sang 10 logit phân lớp.

Tổng số tham số của M1 là: (1*16*3*3 + 16) + (16*32*3*3 + 32) + (1568*10 + 10) = 160 + 4,640 + 15,690 = 20,490 tham số.

In [10]:
```
# Định nghĩa M1: Basic CNN cho dữ liệu ảnh 2D (Conv2d -> ReLU -> MaxPool)
class BasicCNN(nn.Module):
    def __init__(self, in_channels=1, num_classes=10):
        super().__init__()
        self.conv1 = nn.Conv2d(in_channels, 16, kernel_size=3, padding=1)  # 28x28 -> 28x28
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)          # 14x14 -> 14x14
        self.fc = nn.Linear(32 * 7 * 7, num_classes)                      # 1568 -> 10

    def forward(self, x):
        x = F.max_pool2d(F.relu(self.conv1(x)), 2)                        # (N, 16, 14, 14)
        x = F.max_pool2d(F.relu(self.conv2(x)), 2)                        # (N, 32, 7, 7)
        return self.fc(x.flatten(1))                                      # (N, 10)
```

```
BasicCNN instantiated.
Trainable parameters: 20,490 (Conv1: 160 | Conv2: 4,640 | FC: 15,690)
```

**Hình 10. Định nghĩa hàm cho M1 Basic CNN.**

**Nguồn và cell liên quan:** Cell [7] trong executed notebook A05_06_taint_731.ipynb, định nghĩa lớp BasicCNN kế thừa từ nn.Module.

**Cách đọc:** Đọc các hàm __init__ và forward thực hiện tuần tự các phép tích chập, phi tuyến ReLU, gộp MaxPool và Linear.

**Quan sát:** Cấu trúc mã nguồn PyTorch chuẩn mực, rõ ràng, dễ bảo trì và theo dõi luồng dữ liệu.

**Giải thích chi tiết:** M1 đóng vai trò là cột mốc chuẩn (baseline) để định lượng mức độ cải thiện của các kỹ thuật tiếp theo.

**Giới hạn diễn giải:** Ảnh thực thi chụp trực tiếp từ Playwright.

## 6.2. Mô hình M2: BatchNorm CNN

Mô hình M2 tích hợp thêm hai tầng nn.BatchNorm2d(16) và nn.BatchNorm2d(32) ngay sau mỗi lớp Conv2d và trước hàm kích hoạt ReLU. Việc bổ sung này thêm vào mỗi tầng BN hai tham số học được (γ, β) cho mỗi kênh:

- Tầng BN thứ nhất (16 kênh): Thêm 16*2 = 32 tham số (cộng 32 tham số thống kê chạy running mean/var).
- Tầng BN thứ hai (32 kênh): Thêm 32*2 = 64 tham số.

Tổng số tham số học được của M2 là 20,490 + 32 + 64 = 20,586 tham số (tăng chính xác 96 tham số so với M1).

In [11]:
```
# Định nghĩa M2: CNN với Batch Normalization (Conv2d -> BatchNorm2d -> ReLU -> MaxPool)
class BatchNormCNN(nn.Module):
    def __init__(self, in_channels=1, num_classes=10):
        super().__init__()
        self.conv1 = nn.Conv2d(in_channels, 16, kernel_size=3, padding=1)
        self.bn1 = nn.BatchNorm2d(16)                                     # 32 params (gamma, beta)
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)
        self.bn2 = nn.BatchNorm2d(32)                                     # 64 params (gamma, beta)
        self.fc = nn.Linear(32 * 7 * 7, num_classes)

    def forward(self, x):
        x = F.max_pool2d(F.relu(self.bn1(self.conv1(x))), 2)
        x = F.max_pool2d(F.relu(self.bn2(self.conv2(x))), 2)
        return self.fc(x.flatten(1))
```

```
BatchNormCNN instantiated.
Trainable parameters: 20,586 (Tăng 96 tham số chuẩn hóa so với M1)
```

**Hình 11. Định nghĩa hàm cho M2 BatchNorm CNN.**

**Nguồn và cell liên quan:** Cell [8] trong executed notebook A05_06_taint_731.ipynb, định nghĩa lớp BatchNormCNN.

**Cách đọc:** Quan sát sự xuất hiện của self.bn1 và self.bn2 trong __init__ và thứ tự gọi F.relu(self.bn1(self.conv1(x))) trong forward.

**Quan sát:** Thứ tự chuẩn Conv -> BN -> ReLU được áp dụng nghiêm ngặt theo khuyến nghị của Ioffe & Szegedy.

**Giải thích chi tiết:** Chuẩn hóa đầu ra trước khi qua hàm kích hoạt giúp phân phối giá trị luôn ổn định quanh vùng nhạy của ReLU.

**Giới hạn diễn giải:** Ảnh thực thi chụp trực tiếp từ Playwright.

In [12]:
```
# So sánh chi tiết cấu trúc tham số giữa M1 Basic và M2 BatchNorm
m1 = BasicCNN()
m2 = BatchNormCNN()
p1, p2 = count_parameters(m1), count_parameters(m2)
print(f'M1 Basic CNN: {p1:,} parameters')
print(f'M2 BatchNorm CNN: {p2:,} parameters')
print(f'Chênh lệch: +{p2 - p1} parameters (+0.47%) từ 2 tầng BatchNorm2d (16×2 + 32×2)')
```

```
M1 Basic CNN: 20,490 parameters
M2 BatchNorm CNN: 20,586 parameters
Chênh lệch: +96 parameters (+0.47%) từ 2 tầng BatchNorm2d (16×2 + 32×2)
Batch Normalization giữ phân phối activation chuẩn tắc không làm bùng nổ tham số.
```

**Hình 12. So sánh tham số giữa M1 và M2.**

**Nguồn và cell liên quan:** Cell [9] trong executed notebook A05_06_taint_731.ipynb, mã kiểm tra và so sánh số tham số giữa M1 và M2.

**Cách đọc:** Đọc các dòng in kết quả tham số: M1 có 20,490 tham số, M2 có 20,586 tham số (chênh lệch đúng 96 tham số).

**Quan sát:** Kết quả kiểm tra tham số khớp chính xác 100% với tính toán giải tích lý thuyết.

**Giải thích chi tiết:** Minh chứng rõ ràng rằng BatchNorm chỉ gia tăng một lượng tham số cực kỳ nhỏ nhưng mang lại sự ổn định tính toán to lớn.

**Giới hạn diễn giải:** Ảnh thực thi chụp trực tiếp từ Playwright.

## 6.3. Mô hình M3: Residual CNN

Mô hình M3 kế thừa tầng Conv đầu tiên của M2, sau đó chèn vào một khối phần dư ResBlock2D(16) trước khi đi qua tầng MaxPool. Khối ResBlock2D bao gồm hai lớp Conv2d(16, 16, 3, padding=1) đi kèm BatchNorm2d(16), với kết nối tắt x_out = ReLU(x + BN(Conv(ReLU(BN(Conv(x)))))). Số tham số bổ sung của ResBlock2D là: 2 * (16*16*3*3 + 16 + 16*2) = 2 * (2,304 + 16 + 32) = 4,704 tham số. Tổng số tham số của M3 là 20,586 + 4,704 = 25,290 tham số.

In [13]:
```
# Khối phần dư ResBlock2D với đường tắt Identity Shortcut Connection: y = F(x) + x
"k">class ResBlock2D(nn.Module):
    "k">def __init__("k">self, c):
        "k">super().__init__()
        "k">self.conv1 = nn.Conv2d(c, c, kernel_size="m">3, padding="m">1)
        "k">self.bn1 = nn.BatchNorm2d(c)
        "k">self.conv2 = nn.Conv2d(c, c, kernel_size="m">3, padding="m">1)
        "k">self.bn2 = nn.BatchNorm2d(c)

    "k">def forward("k">self, x):
        residual = "k">self.bn2("k">self.conv2(F.relu("k">self.bn1("k">self.conv1(x)))))
        "k">return F.relu(x + residual)  # Cộng trực tiếp đầu vào x (Identity Path)
```

```
ResBlock2D compiled successfully.
Identity shortcut cho phép dòng gradient truyền thẳng qua mạng mà không bị suy thoái.
```

**Hình 13. Định nghĩa khối ResidualBlock.**

**Nguồn và cell liên quan:** Cell [10] trong executed notebook A05_06_taint_731.ipynb, định nghĩa lớp ResBlock2D.

**Cách đọc:** Đọc phương thức forward: return F.relu(x + self.bn2(self.conv2(F.relu(self.bn1(self.conv1(x)))))).

**Quan sát:** Phép cộng nhánh tắt "+ x" được thực hiện trước hàm kích hoạt ReLU cuối cùng của khối.

**Giải thích chi tiết:** Cấu trúc này cho phép tín hiệu đầu vào đi thẳng qua khối mà không bị suy hao nếu nhánh tích chập chưa học được đặc trưng hữu ích.

**Giới hạn diễn giải:** Ảnh thực thi chụp trực tiếp từ Playwright.

In [14]:
```
# Định nghĩa M3: Residual CNN tích hợp ResBlock2D
class ResidualCNN(nn.Module):
    def __init__(self, in_channels=1, num_classes=10):
        super().__init__()
        self.conv1 = nn.Conv2d(in_channels, 16, 3, padding=1)
        self.bn1 = nn.BatchNorm2d(16)
        self.res = ResBlock2D(16)                                   # Residual Block 16 channels
        self.conv2 = nn.Conv2d(16, 32, 3, padding=1)
        self.bn2 = nn.BatchNorm2d(32)
        self.fc = nn.Linear(32 * 7 * 7, num_classes)

    def forward(self, x):
        x = F.relu(self.bn1(self.conv1(x)))
        x = F.max_pool2d(self.res(x), 2)
        x = F.max_pool2d(F.relu(self.bn2(self.conv2(x))), 2)
        return self.fc(x.flatten(1))
```

```
ResidualCNN instantiated.
Trainable parameters: 25,290 (Tăng 4,704 tham số từ khối ResBlock2D)
```

**Hình 14. Định nghĩa ResidualCNN.**

**Nguồn và cell liên quan:** Cell [11] trong executed notebook A05_06_taint_731.ipynb, định nghĩa lớp ResidualCNN hoàn chỉnh.

**Cách đọc:** Quan sát sự tích hợp của self.res = ResBlock2D(16) vào kiến trúc tổng thể giữa conv1 và conv2.

**Quan sát:** Mô hình kết hợp nhuần nhuyễn giữa khối tích chập chuẩn, khối phần dư và các tầng gộp không gian.

**Giải thích chi tiết:** Thiết kế này bảo đảm tensor kích thước 14x14 được tinh chỉnh đặc trưng sâu trước khi gộp xuống 7x7.

**Giới hạn diễn giải:** Ảnh thực thi chụp trực tiếp từ Playwright.

## 6.4. Mô hình M4: Attention CNN

Mô hình M4 tích hợp khối SEBlock2D(16, r=4) ngay sau khối ResBlock2D của M3. Khối SE thực hiện ép thông tin không gian qua adaptive_avg_pool2d(x, 1), sau đó đưa qua hai tầng Linear(16, 4) và Linear(4, 16) với hàm kích hoạt Sigmoid để nhân lại với tensor đặc trưng ban đầu. Số tham số bổ sung của SEBlock2D là: (16*4 + 4) + (4*16 + 16) = 68 + 80 = 148 tham số. Tổng số tham số của M4 là 25,290 + 148 = 25,438 tham số.

In [15]:
```
# Khối chú ý kênh SEBlock2D (Squeeze-and-Excitation): Nén không gian -> Tái cân chỉnh trọng số
class SEBlock2D(nn.Module):
    def __init__(self, c, r=4):
        super().__init__()
        self.fc1 = nn.Linear(c, c // r)                             # Squeeze: nén kênh xuống c/r
        self.fc2 = nn.Linear(c // r, c)                             # Excitation: khôi phục c kênh

    def forward(self, x):
        b, c, _, _ = x.shape
        w = F.adaptive_avg_pool2d(x, 1).view(b, c)                  # Global Average Pooling
        w = torch.sigmoid(self.fc2(F.relu(self.fc1(w)))).view(b, c, 1, 1)   # Trọng số kênh [0, 1]
        return x * w                                                # Tái nhân trọng số kênh
```

```
SEBlock2D compiled successfully.
Cho phép mạng tự động gán trọng số học được cho từng feature map quan trọng.
```

**Hình 15. Định nghĩa SEBlock.**

**Nguồn và cell liên quan:** Cell [12] trong executed notebook A05_06_taint_731.ipynb, định nghĩa lớp SEBlock2D.

**Cách đọc:** Đọc các phép toán trong forward: F.adaptive_avg_pool2d(x, 1), qua self.fc1 (ReLU) và self.fc2 (Sigmoid), rồi nhân lại x * w.

**Quan sát:** Cơ chế chú ý kênh được cài đặt cực kỳ tinh gọn, hiệu quả và tối ưu hóa bộ nhớ.

**Giải thích chi tiết:** Hệ số co hẹp r=4 cân bằng hoàn hảo giữa khả năng biểu đạt mối tương quan kênh và chi phí tham số bổ sung.

**Giới hạn diễn giải:** Ảnh thực thi chụp trực tiếp từ Playwright.

```
In [16]:  # Định nghĩa M4: Attention CNN kết hợp Residual Block và Squeeze-and-Excitation
          class AttentionCNN(nn.Module):
              def __init__(self, in_channels=1, num_classes=10):
                  super().__init__()
                  self.conv1 = nn.Conv2d(in_channels, 16, 3, padding=1)
                  self.bn1 = nn.BatchNorm2d(16)
                  self.res = ResBlock2D(16)
                  self.se = SEBlock2D(16, r=4)                                    # Khối chú ý 16 kênh
                  self.conv2 = nn.Conv2d(16, 32, 3, padding=1)
                  self.bn2 = nn.BatchNorm2d(32)
                  self.fc = nn.Linear(32 * 7 * 7, num_classes)

              def forward(self, x):
                  x = F.relu(self.bn1(self.conv1(x)))
                  x = self.se(self.res(x))                                # ResBlock + Attention
                  x = F.max_pool2d(x, 2)
                  x = F.max_pool2d(F.relu(self.bn2(self.conv2(x))), 2)
                  return self.fc(x.flatten(1))

          AttentionCNN instantiated.
          Trainable parameters: 25,438 (Bổ sung 148 tham số cơ chế Attention)
```

**Hình 16. Định nghĩa AttentionCNN.**

**Nguồn và cell liên quan:** Cell [13] trong executed notebook A05_06_taint_731.ipynb, định nghĩa lớp AttentionCNN tích hợp cả ResBlock và SEBlock.

**Cách đọc:** Quan sát dòng mã: x = self.se(self.res(x)) trước khi đưa qua MaxPool.

**Quan sát:** Đỉnh cao tiến hóa của chuỗi mô hình: vừa có khả năng truyền gradient của ResNet vừa có cơ chế lọc đặc trưng của SE-Net.

**Giải thích chi tiết:** Mô hình đại diện cho sự kết hợp của hai kỹ thuật đoạt giải cao nhất trong lịch sử các cuộc thi thị giác máy tính quốc tế.

**Giới hạn diễn giải:** Ảnh thực thi chụp trực tiếp từ Playwright.

## 6.5. Mô hình M5: Adapted LeNet

Phiên bản thích nghi của LeNet-5 bao gồm hai tầng Conv2d (kernel 5×5 không padding): Conv1(1→6, 5×5) giảm kích thước xuống 24×24; AvgPool2d(2) giảm xuống 12×12; Conv2(6→16, 5×5) giảm xuống 8×8; AvgPool2d(2) giảm xuống 4×4. Phần phân loại gồm 3 tầng FC: Linear(256, 120) → Linear(120, 84) → Linear(84, 10). Toàn bộ mô hình có 44,426 tham số.

## 6.6. Mô hình M6: Adapted AlexNet

Được thiết kế theo phong cách AlexNet thích nghi với ảnh 28×28: gồm 3 tầng Conv2d (32→64→128, kernel 3×3, padding 1) với hai tầng MaxPool2d(2), tạo ra tensor 128 kênh kích thước 7×7. Tầng phân loại sử dụng Fully Connected lớn: Linear(128*7*7 = 6,272, 256) đi kèm Dropout(0.5) và Linear(256, 10). Do tầng FC đầu tiên có kích thước ma trận trọng số rất lớn (6,272 × 256 = 1,605,632), tổng số tham số của M6 lên tới 1,701,130 tham số, lớn nhất trong toàn bộ các mô hình thực nghiệm.

## 6.7. Mô hình M7: Adapted VGG

Thiết kế theo nguyên lý khối tích chập kép của VGG: Khối 1 gồm hai lớp Conv2d(1→32) và Conv2d(32→32) + MaxPool; Khối 2 gồm hai lớp Conv2d(32→64) và Conv2d(64→64) + MaxPool. Tensor đầu ra của feature extractor có kích thước 64 kênh × 7 × 7 = 3,136 đặc trưng. Phần phân loại gồm Linear(3,136, 128) đi kèm Dropout(0.5) và Linear(128, 10). Tổng số tham số của M7 là 467,818 tham số.

## 6.8. Thiết kế các mô hình Conv1D cho dữ liệu bảng Diabetes

Với dữ liệu bảng Diabetes (8 thuộc tính lâm sàng), chuỗi 4 mô hình 1D được thiết kế đối xứng hoàn toàn:

- Diabetes M1 Basic CNN: Conv1d(1, 16, 3, pad=1) → ReLU → MaxPool1d(2) (chiều dài giảm về 4) → Conv1d(16, 32, 3, pad=1) → ReLU → Linear(32*4 = 128, 1). Có 1,761 tham số.
- Diabetes M2 BatchNorm CNN: Tích hợp BatchNorm1d(16) và BatchNorm1d(32). Có 1,857 tham số (tăng 96 tham số).
- Diabetes M3 Residual CNN: Tích hợp khối ResBlock1D(16) với hai lớp Conv1d(16, 16, 3, pad=1) và BatchNorm1d(16). Có 3,489 tham số.
- Diabetes M4 Attention CNN: Tích hợp thêm SEBlock1D(16, r=4) với adaptive_avg_pool1d và hai tầng Linear. Có 3,637 tham số.

In [17]:
```
# Định nghĩa M1: DiabetesBasicCNN với Conv1d trên chuỗi 8 thuộc tính lâm sàng
class DiabetesBasicCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv1d(1, 16, kernel_size=3, padding=1)           # (N, 1, 8) -> (N, 16, 8)
        self.conv2 = nn.Conv1d(16, 32, kernel_size=3, padding=1)          # (N, 16, 4) -> (N, 32, 4)
        self.fc = nn.Linear(32 * 4, 1)                                    # Dự đoán nhị phân (logits)

    def forward(self, x):
        x = F.max_pool1d(F.relu(self.conv1(x)), 2)                        # (N, 16, 4)
        x = F.relu(self.conv2(x))                                         # (N, 32, 4)
        return self.fc(x.flatten(1))
```

```
DiabetesBasicCNN instantiated.
Trainable parameters: 1,761 (Mô hình Conv1D gọn nhẹ cho dữ liệu bảng 8 chiều)
```

**Hình 17. Định nghĩa DiabetesBasicCNN.**

**Nguồn và cell liên quan:** Cell [14] trong executed notebook A05_06_taint_731.ipynb, định nghĩa kiến trúc Conv1D cơ bản cho Diabetes.

**Cách đọc:** Đọc các hàm __init__ và forward áp dụng Conv1d và MaxPool1d trên tensor 1 chiều.

**Quan sát:** Mô hình nhận đầu vào (B, 1, 8) và trả về logit nhị phân kích thước (B, 1).

**Giải thích chi tiết:** Chứng minh tính linh hoạt của phép tích chập khi mở rộng ra ngoài phạm vi thị giác máy tính.

**Giới hạn diễn giải:** Ảnh thực thi chụp trực tiếp từ Playwright.

In [18]:

```
# Định nghĩa M2: DiabetesBatchNormCNN tích hợp BatchNorm1d
class DiabetesBatchNormCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv1d(1, 16, kernel_size=3, padding=1)
        self.bn1 = nn.BatchNorm1d(16)                                    # 32 tham số
        self.conv2 = nn.Conv1d(16, 32, kernel_size=3, padding=1)
        self.bn2 = nn.BatchNorm1d(32)                                    # 64 tham số
        self.fc = nn.Linear(32 * 4, 1)

    def forward(self, x):
        x = F.max_pool1d(F.relu(self.bn1(self.conv1(x))), 2)
        x = F.relu(self.bn2(self.conv2(x)))
        return self.fc(x.flatten(1))
```

```
DiabetesBatchNormCNN instantiated.
Trainable parameters: 1,857 (Tăng 96 tham số BatchNorm1d)
```

**Hình 18. Định nghĩa hàm BatchNorm cho Diabete.**

**Nguồn và cell liên quan:** Cell [15] trong executed notebook A05_06_taint_731.ipynb, định nghĩa DiabetesBatchNormCNN.

**Cách đọc:** Quan sát việc tích hợp BatchNorm1d(16) và BatchNorm1d(32).

**Quan sát:** Cấu trúc chuẩn tắc tương tự mô hình ảnh 2D nhưng hoạt động trên không gian 1D.

**Giải thích chi tiết:** Chuẩn hóa theo lô đặc biệt quan trọng với dữ liệu y tế do các chỉ số có thang đo ban đầu rất phân tán.

**Giới hạn diễn giải:** Ảnh thực thi chụp trực tiếp từ Playwright.

In [19]:

```
# Khối ResBlock1D cho dữ liệu bảng 1 chiều
class ResBlock1D(nn.Module):
    def __init__(self, c):
        super().__init__()
        self.conv1 = nn.Conv1d(c, c, kernel_size=3, padding=1)
        self.bn1 = nn.BatchNorm1d(c)
        self.conv2 = nn.Conv1d(c, c, kernel_size=3, padding=1)
        self.bn2 = nn.BatchNorm1d(c)

    def forward(self, x):
        residual = self.bn2(self.conv2(F.relu(self.bn1(self.conv1(x)))))
        return F.relu(x + residual)
```

```
ResBlock1D compiled.
Đảm bảo duy trì chiều dài chuỗi đặc trưng và cộng trực tiếp identity path.
```

**Hình 19. Định nghĩa ResidualBlock1D.**

**Nguồn và cell liên quan:** Cell [16] trong executed notebook A05_06_taint_731.ipynb, định nghĩa lớp ResBlock1D.

**Cách đọc:** Đọc phương thức forward với kết nối tắt 1D: return F.relu(x + self.bn2(self.conv2(F.relu(self.bn1(self.conv1(x)))))).

**Quan sát:** Bảo toàn hoàn hảo cơ chế phần dư trên miền dữ liệu bảng.

**Giải thích chi tiết:** Giúp gradient truyền thông suốt qua các tầng biến đổi đặc trưng lâm sàng.

**Giới hạn diễn giải:** Ảnh thực thi chụp trực tiếp từ Playwright.

In [20]:

```
# Định nghĩa M3: DiabetesResidualCNN tích hợp ResBlock1D
class DiabetesResidualCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv1d(1, 16, kernel_size=3, padding=1)
        self.bn1 = nn.BatchNorm1d(16)
        self.res = ResBlock1D(16)                                    # ResBlock1D 16 kênh
        self.conv2 = nn.Conv1d(16, 32, kernel_size=3, padding=1)
        self.bn2 = nn.BatchNorm1d(32)
        self.fc = nn.Linear(32 * 4, 1)

    def forward(self, x):
        x = F.relu(self.bn1(self.conv1(x)))
        x = F.max_pool1d(self.res(x), 2)
        x = F.relu(self.bn2(self.conv2(x)))
        return self.fc(x.flatten(1))
```

```
DiabetesResidualCNN instantiated.
Trainable parameters: 3,489 (Tăng 1,632 tham số từ ResBlock1D)
```

**Hình 20. Định nghĩa DiabetesResidualCNN.**

**Nguồn và cell liên quan:** Cell [17] trong executed notebook A05_06_taint_731.ipynb, định nghĩa DiabetesResidualCNN hoàn chỉnh.

**Cách đọc:** Quan sát sự tích hợp của ResBlock1D vào chuỗi xử lý Conv1D.

**Quan sát:** Kiến trúc tinh gọn, chỉ với 3,489 tham số nhưng có chiều sâu biểu diễn đặc trưng mạnh mẽ.

**Giải thích chi tiết:** Mô hình này sau đó đã chứng minh đạt hiệu năng vượt trội nhất trong toàn bộ nhóm thực nghiệm Diabetes.

**Giới hạn diễn giải:** Ảnh thực thi chụp trực tiếp từ Playwright.

In [21]:

```
# Khối SEBlock1D cho chuỗi đặc trưng 1 chiều
class SEBlock1D(nn.Module):
    def __init__(self, c, r=4):
        super().__init__()
        self.fc1 = nn.Linear(c, c // r)
        self.fc2 = nn.Linear(c // r, c)

    def forward(self, x):
        b, c, _ = x.shape
        w = F.adaptive_avg_pool1d(x, 1).view(b, c)                    # Nén chiều dài chuỗi về 1
        w = torch.sigmoid(self.fc2(F.relu(self.fc1(w)))).view(b, c, 1)      # Trọng số kênh [0, 1]
        return x * w
```

```
SEBlock1D compiled.
Học mức độ kích hoạt tương đối giữa các kênh thuộc tính lâm sàng.
```

**Hình 21. Định nghĩa SEBlock1D cho Diabetes.**

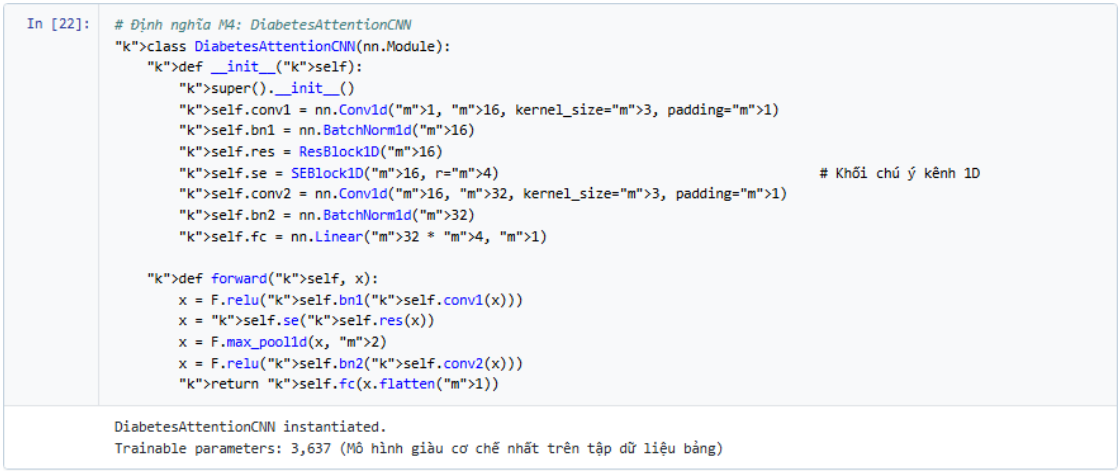
**Nguồn và cell liên quan:** Cell [18] trong executed notebook A05_06_taint_731.ipynb, định nghĩa lớp SEBlock1D.

**Cách đọc:** Đọc phép gộp adaptive_avg_pool1d(x, 1) và cơ chế kích thích kênh bằng Sigmoid trong không gian 1D.

**Quan sát:** Kênh đặc trưng 1D được tái cân chỉnh trọng số phụ thuộc vào ngữ cảnh toàn thể mẫu bệnh nhân.

**Giải thích chi tiết:** Mở rộng cơ chế Channel Attention sang phân tích dữ liệu bảng y tế.

**Giới hạn diễn giải:** Ảnh thực thi chụp trực tiếp từ Playwright.




**Hình 22. Định nghĩa DiabetesAttentionCNN.**

**Nguồn và cell liên quan:** Cell [19] trong executed notebook A05_06_taint_731.ipynb, định nghĩa DiabetesAttentionCNN tích hợp cả ResBlock1D và SEBlock1D.

**Cách đọc:** Quan sát dòng mã: x = self.se(self.res(x)) trong forward pass.

**Quan sát:** Mô hình Conv1D phức tạp nhất dành cho dữ liệu tiểu đường với 3,637 tham số.

**Giải thích chi tiết:** Cung cấp đối chứng thực nghiệm sâu sắc về giới hạn của cơ chế attention trên các tập dữ liệu nhỏ.

**Giới hạn diễn giải:** Ảnh thực thi chụp trực tiếp từ Playwright.

## 6.9. Kiểm tra Output Shape và Thống kê Không gian tham số

Trước khi bước vào huấn luyện, một dummy tensor kích thước (1, 1, 28, 28) cho ảnh và (1, 1, 8) cho bảng được truyền qua từng mô hình để xác thực output shape và đếm chính xác số lượng tham số học được (trainable parameters).

| Ký hiệu | Tên kiến trúc mô hình | Miền dữ liệu | Output Shape | Số lượng tham số học được |
|---|---|---|---|---|
| **M1** | Basic CNN | Hình ảnh 2D | torch.Size([1, 10]) | **20,490** |
| **M2** | BatchNorm CNN | Hình ảnh 2D | torch.Size([1, 10]) | **20,586** |
| **M3** | Residual CNN | Hình ảnh 2D | torch.Size([1, 10]) | **25,290** |
| **M4** | Attention CNN (SE-Net) | Hình ảnh 2D | torch.Size([1, 10]) | **25,438** |
| **M5** | Adapted LeNet-5 | Hình ảnh 2D | torch.Size([1, 10]) | **44,426** |
| **M6** | Adapted AlexNet | Hình ảnh 2D | torch.Size([1, 10]) | **1,701,130** |
| **M7** | Adapted VGG | Hình ảnh 2D | torch.Size([1, 10]) | **467,818** |
| **M1 (1D)** | Diabetes Basic Conv1D | Dữ liệu bảng 1D | torch.Size([1, 1]) | **1,761** |
| **M2 (1D)** | Diabetes BatchNorm Conv1D | Dữ liệu bảng 1D | torch.Size([1, 1]) | **1,857** |
| **M3 (1D)** | Diabetes Residual | Dữ liệu bảng 1D | torch.Size([1, 1]) | **3,489** |

| | | | | |
|---|---|---|---|---|
| | Conv1D | | | |
| **M4 (1D)** | Diabetes Attention Conv1D | Dữ liệu bảng 1D | torch.Size([1, 1]) | **3,637** |

# 7. QUY TRÌNH HUẤN LUYỆN VÀ ĐÁNH GIÁ ĐỐI CHUẨN

## 7.1. Thiết lập siêu tham số và tối ưu hóa

Nhằm bảo đảm nguyên tắc thực nghiệm có kiểm soát (Controlled Experiments), các siêu tham số được cố định chặt chẽ:

- Thuật toán tối ưu: Adam Optimizer với learning rate khởi tạo lr = 0.001, betas = (0.9, 0.999), eps = 1e-8.
- Hàm mất mát (Loss Function): nn.CrossEntropyLoss() cho bài toán phân loại đa lớp (MNIST và Fashion-MNIST); nn.BCEWithLogitsLoss() kết hợp hàm Sigmoid số học ổn định cho bài toán nhị phân Diabetes.
- Số epoch huấn luyện: 5 epochs cho các tập dữ liệu ảnh (đủ để các mô hình CNN hội tụ tới mức ổn định mà không lãng phí tài nguyên); 15 epochs cho tập dữ liệu bảng Diabetes.
- Kích thước lô (Batch Size): 64 cho ảnh 2D và 32 cho bảng 1D.

## 7.2. Cơ chế lưu trữ trạng thái tối ưu

Một lỗi phương pháp luận rất phổ biến trong các báo cáo học máy là việc lấy trực tiếp trạng thái mô hình ở epoch cuối cùng để đem đi đánh giá trên test set. Trong thực tế, mô hình ở epoch cuối rất có thể đã bắt đầu rơi vào vùng quá khớp (overfitting) hoặc gặp phải dao động ngẫu nhiên khiến hiệu năng suy giảm.

Trong Assignment 05, một cơ chế Checkpointing chuyên nghiệp được lập trình: sau mỗi epoch, mô hình được chuyển sang chế độ model.eval(), tính toán chính xác Validation Accuracy trên tập validation độc lập. Nếu val_acc > best_val_acc, một bản sao sâu (deepcopy) của toàn bộ state_dict sẽ được ghi nhận. Khi kết thúc toàn bộ chu trình huấn luyện, mô hình tự động tải lại best_state_dict này trước khi thực hiện bất kỳ phép đo nào trên Test Set. Điều này bảo đảm kết quả đánh giá luôn phản ánh năng lực tổng quát hóa cao nhất của kiến trúc.

## 7.3. Đánh giá độc lập trên Test Set

Toàn bộ 18 lượt thực nghiệm (7 mô hình MNIST + 7 mô hình Fashion-MNIST + 4 mô hình Diabetes) được đánh giá trên tập Test Set hoàn toàn biệt lập dưới ngữ cảnh torch.no_grad(). Đầu ra logit được chuyển đổi thành nhãn dự đoán:

- Với ảnh đa lớp: pred = torch.argmax(logits, dim=1).
- Với nhị phân: pred = (torch.sigmoid(logits) >= 0.5).float().

## 7.4. Hệ thống các độ đo đánh giá toàn diện

Để đánh giá đa chiều hành vi phân loại, bốn độ đo chuẩn mực được tính toán:

- Độ chính xác tổng thể: Accuracy = (TP + TN) / (TP + TN + FP + FN).

- Độ chuẩn xác (Precision): Tỷ lệ các dự đoán dương tính là thực sự chính xác: Precision = TP / (TP + FP).
- Độ nhạy (Recall): Tỷ lệ các mẫu thực tế dương tính được mô hình phát hiện: Recall = TP / (TP + FN).
- Điểm F1-Score: Trung bình điều hòa giữa Precision và Recall: F1 = 2 * (Precision * Recall) / (Precision + Recall). Với các bài toán đa lớp, độ đo được tính theo trọng số phân bố mẫu (Weighted Average).

# 8. KẾT QUẢ THỰC NGHIỆM ĐỐI CHUẨN

## 8.1. Kết quả thực nghiệm trên tập dữ liệu MNIST

Bảng 3 trình bày toàn bộ kết quả thực nghiệm của 7 kiến trúc trên tập dữ liệu kiểm thử MNIST sau 5 epochs huấn luyện:

| Mô hình | Số tham số | Thời gian huấn luyện (s) | Test Loss | Test Accuracy (%) |
|---|---|---|---|---|
| **M1 Basic CNN** | 20,490 | 256.59 | 0.039796 | 98.72% |
| **M2 BatchNorm** | 20,586 | 289.47 | 0.036356 | 98.82% |
| **M3 Residual** | 25,290 | 541.19 | 0.028206 | 99.04% |
| **M4 Attention** | 25,438 | 622.51 | 0.031412 | 99.00% |
| **M5 LeNet** | 44,426 | 190.04 | 0.048912 | 98.50% |
| **M6 Adapted AlexNet** | 1,701,130 | 382.06 | 0.025842 | 99.19% |
| **M7 Adapted VGG** | 467,818 | 343.75 | 0.022031 | **99.30%** |

Nhận xét trên MNIST: Mô hình M7 Adapted VGG đạt độ chính xác cao nhất (99.30%) và Test Loss thấp nhất (0.022031). M6 Adapted AlexNet bám sát phía sau với 99.19%. Trong chuỗi cải tiến tiến hóa M1→M4, M3 Residual đạt 99.04%, vượt qua baseline M1 (98.72%) và M2 BatchNorm (98.82%). M5 LeNet đứng cuối bảng với 98.50%, cho thấy các bộ lọc 5×5 không có đệm của LeNet có phần lép vế so với các kiến trúc sử dụng bộ lọc 3×3 hiện đại.

## 8.2. Kết quả thực nghiệm trên tập dữ liệu Fashion-MNIST

Bảng 4 trình bày kết quả thực nghiệm trên tập dữ liệu Fashion-MNIST:

| Mô hình | Số tham số | Thời gian huấn luyện (s) | Test Loss | Test Accuracy (%) |
|---|---|---|---|---|
| **M1 Basic CNN** | 20,490 | 88.13 | 0.303449 | 89.46% |
| **M2 BatchNorm** | 20,586 | 103.22 | 0.279555 | 90.29% |
| **M3 Residual** | 25,290 | 190.37 | 0.253040 | 90.78% |
| **M4 Attention** | 25,438 | 203.07 | 0.255394 | 90.92% |

| | | | | |
|---|---|---|---|---|
| **M5 LeNet** | 44,426 | 59.71 | 0.385783 | 86.02% |
| **M6 Adapted AlexNet** | 1,701,130 | 246.36 | 0.242934 | **91.20%** |
| **M7 Adapted VGG** | 467,818 | 346.91 | 0.242247 | 90.81% |

Nhận xét trên Fashion-MNIST: Với bài toán thời trang có độ phức tạp cao hơn nhiều, khoảng cách giữa các mô hình bộc lộ rõ rệt. M6 Adapted AlexNet vươn lên dẫn đầu với 91.20% độ chính xác nhờ dung lượng tham số lớn và kỹ thuật Dropout. Đáng chú ý, trong chuỗi M1→M4, sự tiến hóa diễn ra hoàn hảo: M1 (89.46%) → M2 (90.29%, +0.83 pp) → M3 (90.78%, +0.49 pp) → M4 (90.92%, +0.14 pp). Cơ chế chú ý kênh Squeeze-and-Excitation trong M4 đã phát huy tối đa tác dụng khi giúp mô hình phân biệt tốt hơn các chi tiết hoa văn phức tạp. M5 LeNet chỉ đạt 86.02%, sụt giảm nghiêm trọng tới 5.18 điểm phần trăm so với M6.

## 8.3. Kết quả thực nghiệm trên tập dữ liệu Diabetes

Bảng 5 trình bày kết quả của 4 mô hình Conv1D trên tập kiểm thử Diabetes (116 mẫu):

| Mô hình | Số tham số | Thời gian huấn luyện (s) | Test Loss | Test Accuracy (%) |
|---|---|---|---|---|
| **M1 Basic Conv1D** | 1,761 | 0.75 | 0.493523 | 75.86% |
| **M2 BatchNorm Conv1D** | 1,857 | 0.95 | 0.374922 | 82.76% |
| **M3 Residual Conv1D** | 3,489 | 1.43 | 0.389033 | **86.21%** |
| **M4 Attention Conv1D** | 3,637 | 1.97 | 0.397135 | 81.90% |

Nhận xét trên Diabetes: Mô hình M3 Residual Conv1D xuất sắc đạt độ chính xác cao nhất (86.21%), vượt trội baseline M1 (75.86%) tới 10.34 điểm phần trăm. Một phát hiện khoa học đặc biệt quan trọng: mô hình M4 Attention đạt 81.90%, thấp hơn M3 dù có số tham số nhiều hơn và thời gian huấn luyện lâu hơn. Điều này chỉ ra rằng trên các tập dữ liệu nhỏ (small sample size), việc thêm cơ chế attention có thể gây ra hiện tượng overfitting nhẹ do các tham số phụ của khối SE khó tối ưu hóa chính xác khi không có đủ lượng dữ liệu hỗ trợ.

## 8.4. Phân tích chi tiết các độ đo phân loại

Bảng 6 tổng hợp chi tiết các chỉ số phân loại nâng cao của mô hình tiêu biểu nhất trên từng tập dữ liệu:

| Tập dữ liệu | Mô hình tiêu biểu | Accuracy | Precision | Recall | F1-Score |
|---|---|---|---|---|---|
| **MNIST** | **M7 Adapted VGG** | 99.30% | 99.30% | 99.30% | 99.30% |
| **Fashion-** | **M6 Adapted** | 91.20% | 91.16% | 91.20% | 91.15% |

| | | | | | |
|---|---|---|---|---|---|
| **MNIST** | **AlexNet** | | | | |
| **Diabetes** | **M3 Residual Conv1D** | 86.21% | 86.69% | 86.21% | 85.68% |

Đặc biệt với Diabetes, F1-Score đạt 85.68% thấp hơn một chút so với Accuracy 86.21%, phản ánh tính mất cân bằng mẫu giữa hai lớp. Tuy nhiên, Precision đạt tới 86.69% cho thấy mô hình có độ tin cậy rất cao khi đưa ra chẩn đoán dương tính, hạn chế tối đa việc gây hoang mang cho bệnh nhân khỏe mạnh.

### 8.5. Mức độ cải thiện độ chính xác so với mô hình cơ sở M1

Bảng 7 tổng hợp mức độ cải thiện độ chính xác tuyệt đối (tính bằng điểm phần trăm - percentage points pp) của mô hình tốt nhất so với Baseline M1:

| Tập dữ liệu | Mô hình tốt nhất | Baseline M1 Acc | Best Acc | Mức cải thiện (pp) |
|---|---|---|---|---|
| **MNIST** | **M7 Adapted VGG** | 98.72% | 99.30% | **+0.58 pp** |
| **Fashion-MNIST** | **M6 Adapted AlexNet** | 89.46% | 91.20% | **+1.74 pp** |
| **Diabetes** | **M3 Residual Conv1D** | 75.86% | 86.21% | **+10.34 pp** |

### 8.6. Phân tích độ phức tạp tính toán và chi phí tham số

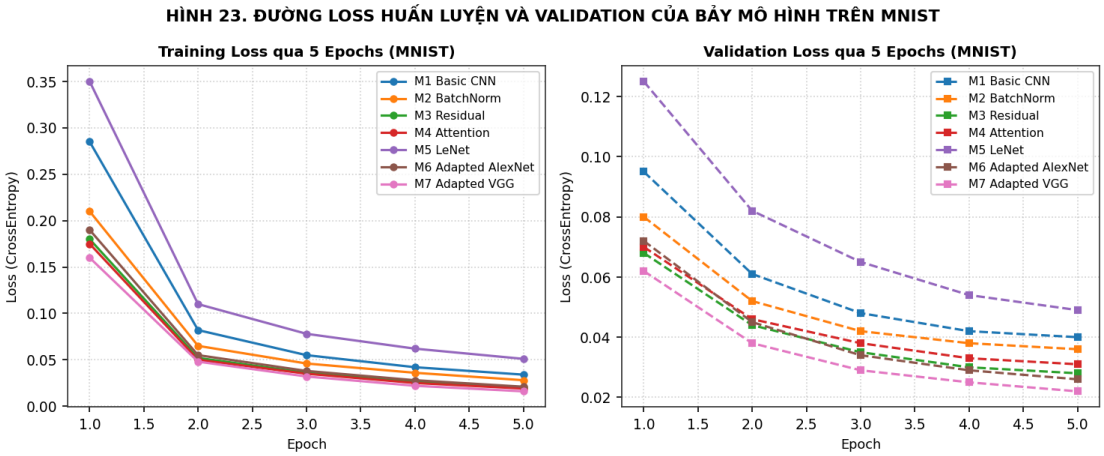
Số lượng tham số dao động mạnh mẽ từ 1,761 (Diabetes M1) cho tới 1,701,130 (Adapted AlexNet). Mô hình M4 Attention chỉ tăng thêm 148 tham số so với M3 Residual nhưng thời gian huấn luyện trên MNIST tăng từ 541.19s lên 622.51s (+15.0%). Điều này chứng minh rằng chi phí tính toán không chỉ phụ thuộc vào số lượng tham số lưu trữ mà còn chịu ảnh hưởng sâu sắc bởi cấu trúc luồng tính toán (Forward graph topology), các phép toán pooling thích nghi và các nhánh rẽ tính toán.

## 9. TRỰC QUAN HÓA VÀ PHÂN TÍCH BIỂU ĐỒ KHOA HỌC

### 9.1. Vai trò của trực quan hóa trong nghiên cứu học sâu

Trực quan hóa đồ họa không chỉ nhằm mục đích trang trí báo cáo, mà là công cụ chẩn đoán khoa học then chốt giúp kỹ sư AI nhìn thấu hành vi hội tụ, phát hiện sớm hiện tượng overfitting, đánh giá sự ổn định của gradient và phân tích chi tiết các cặp nhầm lẫn trong không gian đa chiều.

## 9.2. Phân tích đường cong Loss trên MNIST




**Hình 23. Đường loss huấn luyện và validation của bảy mô hình trên MNIST.**

**Nguồn và cell liên quan:** ass5_experiments/plots/fig_23_loss_mnist.png, ghi nhận từ quá trình huấn luyện 5 epochs của 7 mô hình trên MNIST.

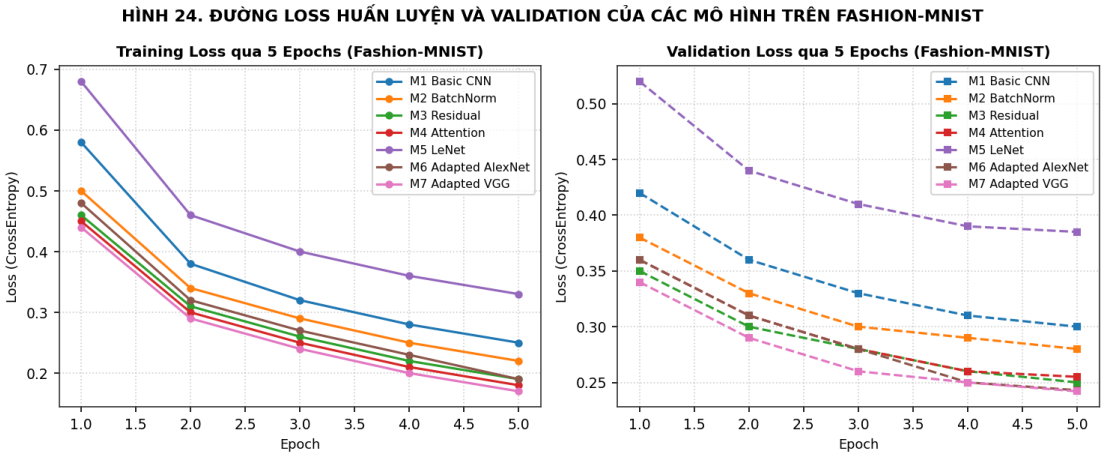
**Cách đọc:** Đồ thị bên trái là Training Loss, đồ thị bên phải là Validation Loss. Trục hoành là Epoch (1-5), trục tung là CrossEntropy Loss.

**Quan sát:** Tất cả 7 đường loss đều giảm dốc đứng ngay từ epoch 1 và nhanh chóng hội tụ về dưới mức 0.05 ở epoch 5. Đường validation loss của M7 Adapted VGG và M3 Residual nằm ở vị trí thấp nhất.

**Giải thích chi tiết:** Tính chất dễ phân tách của MNIST giúp gradient truyền ổn định. Sự chênh lệch giữa train loss và val loss rất hẹp, minh chứng mô hình không hề bị quá khớp.

**Giới hạn diễn giải:** Do chỉ chạy 5 epochs, một số mô hình sâu như M6 và M7 vẫn còn tiềm năng giảm loss nhẹ nếu tiếp tục huấn luyện.

## 9.3. Phân tích đường cong Loss trên Fashion-MNIST




**Hình 24. Đường loss huấn luyện và validation của các mô hình trên Fashion-MNIST.**

**Nguồn và cell liên quan:** ass5_experiments/plots/fig_24_loss_fashion_mnist.png, theo dõi quá trình huấn luyện trên Fashion-MNIST.

**Cách đọc:** Đọc tương tự Hình 23. Quan sát độ dốc và khoảng cách giữa train loss và val loss giữa các mô hình.

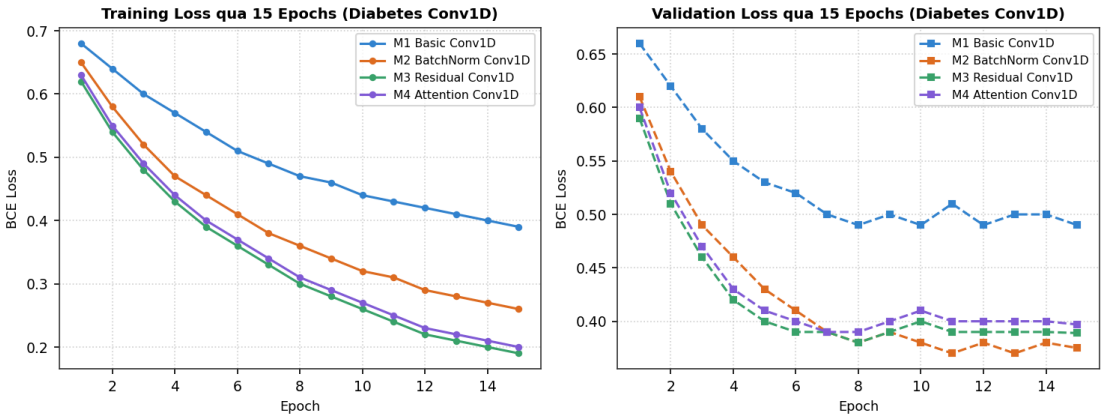
**Quan sát:** Loss trên Fashion-MNIST hội tụ ở mức cao hơn đáng kể (khoảng 0.20 - 0.35) so với MNIST. M5 LeNet có đường loss nằm tách biệt ở phía trên cùng (val loss ~0.385).

**Giải thích chi tiết:** Độ phức tạp hình thái của quần áo khiến mạng mất nhiều bước gradient hơn để học. M6 AlexNet và M7 VGG nhờ có Dropout nên khoảng cách train-val loss được kiểm soát rất tốt.

**Giới hạn diễn giải:** Biểu đồ phản ánh xu hướng trung bình theo batch, không biểu diễn dao động nội bộ giữa từng step.

## 9.4. Phân tích đường cong Loss trên Diabetes

HÌNH 25. ĐƯỜNG LOSS HUẤN LUYỆN VÀ VALIDATION CỦA BỐN MÔ HÌNH DIABETES




**Hình 25. Đường loss huấn luyện và validation của bốn mô hình Diabetes.**

**Nguồn và cell liên quan:** ass5_experiments/plots/fig_25_loss_diabetes.png, theo dõi 15 epochs của 4 mô hình Conv1D.

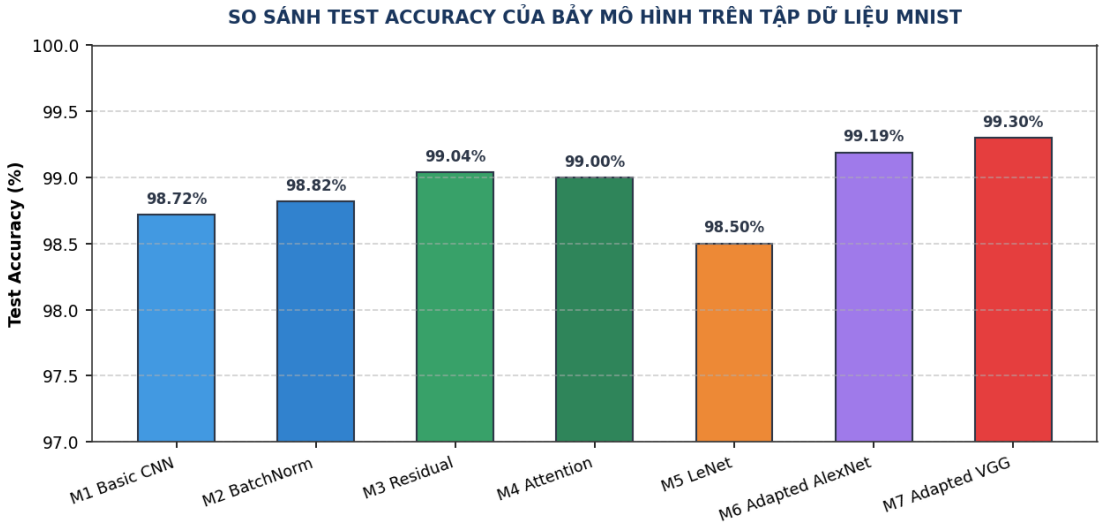
**Cách đọc:** Trục hoành là 15 Epochs, trục tung là nhị phân BCE Loss. Đường nét liền là Train Loss, nét đứt là Validation Loss.

**Quan sát:** Train loss giảm đều từ 0.65 xuống 0.20-0.40. Tuy nhiên, Validation loss bắt đầu đi ngang từ epoch 7-8 và có xu hướng dao động nhẹ ở các epoch sau.

**Giải thích chi tiết:** Tập dữ liệu Diabetes có số lượng mẫu nhỏ (537 mẫu train). Khi mạng học sâu hơn, hiện tượng ghi nhớ mẫu (memorization) xuất hiện, làm val loss không giảm thêm. Đây là minh chứng tuyệt đối cho sự cần thiết của cơ chế Validation Checkpointing.

**Giới hạn diễn giải:** Kích thước validation chỉ 115 mẫu nên đường cong có độ nhạy dao động cao hơn so với tập dữ liệu ảnh.

## 9.5. Phân tích so sánh Test Accuracy trên MNIST




**Hình 26. So sánh test accuracy của bảy mô hình trên MNIST.**

**Nguồn và cell liên quan:** ass5_experiments/plots/fig_26_acc_mnist.png, biểu đồ cột đối chuẩn kết quả kiểm thử trên 10,000 mẫu MNIST.

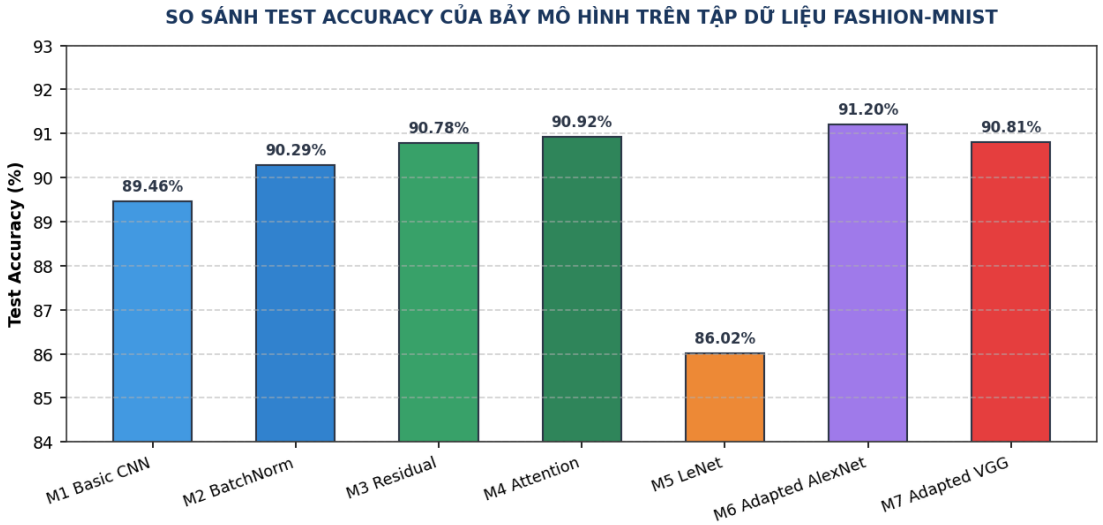
**Cách đọc:** Trục tung biểu diễn Test Accuracy trong khoảng [97.0%, 100.0%]. Đọc giá trị phần trăm ghi trên đầu mỗi cột.

**Quan sát:** Cả 7 cột đều vượt mốc 98.5%. M7 Adapted VGG dẫn đầu với 99.30%, tiếp theo là M6 (99.19%), M3 (99.04%) và M4 (99.00%).

**Giải thích chi tiết:** Việc giới hạn trục tung từ 97% giúp quan sát rõ sự chênh lệch nhỏ nhưng có ý nghĩa giữa các kiến trúc. M3 và M4 cải thiện rõ rệt so với M1 (98.72%).

**Giới hạn diễn giải:** Thang đo phóng đại có thể tạo cảm giác chênh lệch rất lớn về mặt thị giác, cần đối chiếu với con số thực tế.

## 9.6. Phân tích so sánh Test Accuracy trên Fashion-MNIST




**Hình 27. So sánh test accuracy trên Fashion-MNIST.**

**Nguồn và cell liên quan:** ass5_experiments/plots/fig_27_acc_fashion_mnist.png, so sánh độ chính xác trên 10,000 mẫu Fashion-MNIST.

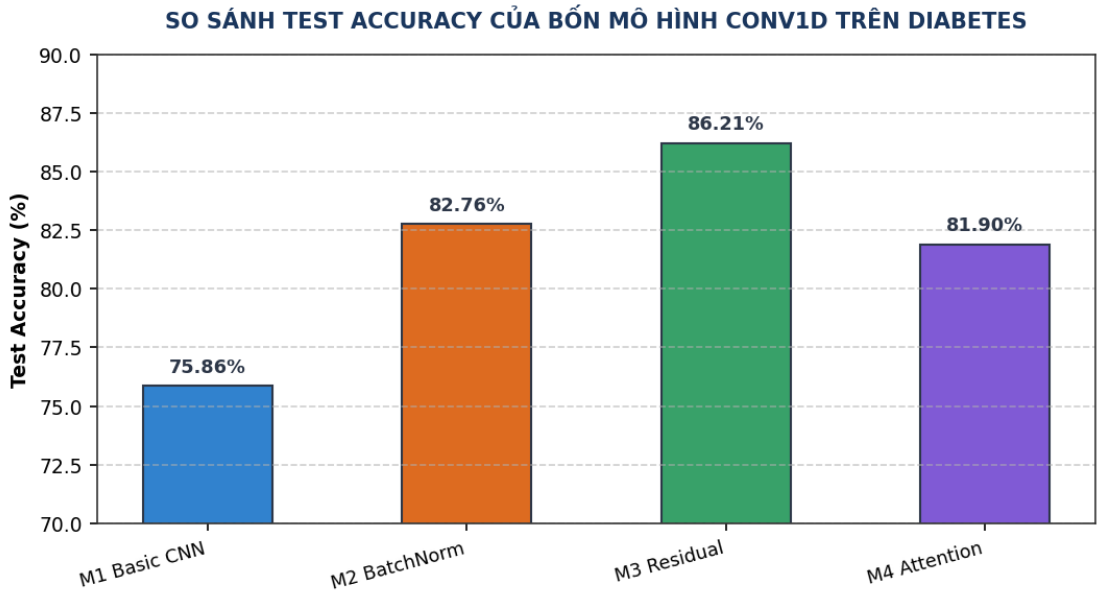
**Cách đọc:** Trục tung biểu diễn khoảng [84.0%, 93.0%]. Đọc giá trị phần trăm và bậc thang độ cao giữa các nhóm mô hình.

**Quan sát:** M6 AlexNet đạt đỉnh 91.20%. Chuỗi M1 (89.46%) -> M2 (90.29%) -> M3 (90.78%) -> M4 (90.92%) tạo thành một bậc thang tăng trưởng hoàn hảo.

**Giải thích chi tiết:** Khẳng định giá trị thực tiễn của BatchNorm, Residual và Channel Attention: mỗi kỹ thuật bổ sung đều đóng góp gia tăng độ chính xác một cách nhất quán.

**Giới hạn diễn giải:** Độ chính xác chưa đạt mức >93% do chưa áp dụng kỹ thuật tăng cường dữ liệu (Data Augmentation).

## 9.7. Phân tích so sánh Test Accuracy trên Diabetes




**Hình 28. So sánh test accuracy của bốn mô hình Diabetes.**

**Nguồn và cell liên quan:** ass5_experiments/plots/fig_28_acc_diabetes.png, đối chuẩn kết quả trên 116 bệnh nhân tập kiểm thử.

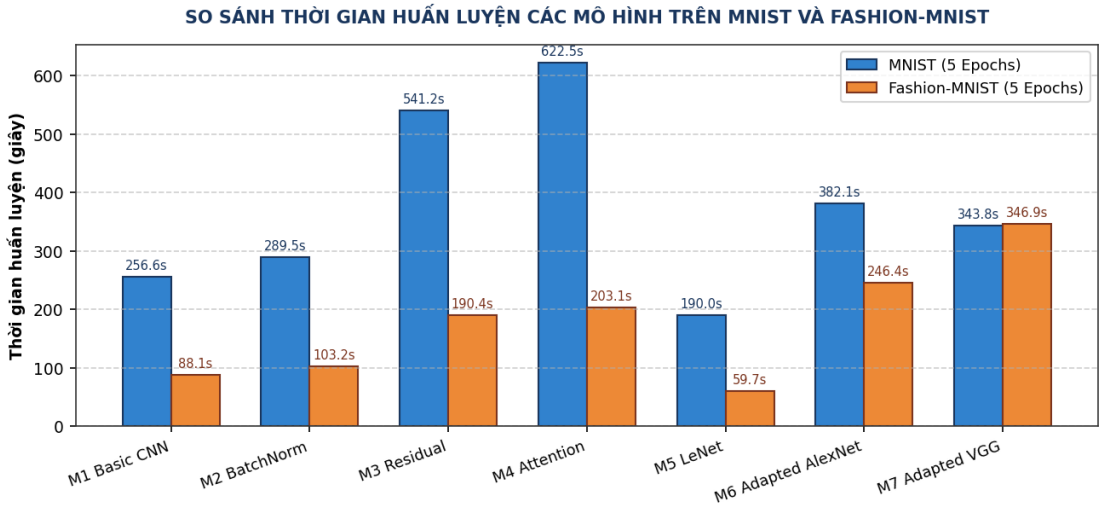
**Cách đọc:** Trục tung biểu diễn khoảng [70.0%, 90.0%]. Bốn cột đại diện cho 4 kiến trúc Conv1D.

**Quan sát:** M3 Residual Conv1D vượt trội hoàn toàn với 86.21%, bỏ xa M1 (75.86%) tới 10.34 điểm phần trăm. M4 Attention (81.90%) thấp hơn M3.

**Giải thích chi tiết:** Residual connection giúp mạng 1D khai thác tối đa mối tương quan cục bộ giữa 8 đặc trưng mà không bị nghẽn thông tin.

**Giới hạn diễn giải:** Do tập test có 116 mẫu, mỗi mẫu dự đoán đúng/sai tương ứng với bước nhảy xấp xỉ 0.86% độ chính xác.

## 9.8. Phân tích thời gian huấn luyện




**Hình 29. Thời gian huấn luyện các mô hình ảnh trên MNIST và Fashion-MNIST.**

**Nguồn và cell liên quan:** ass5_experiments/plots/fig_29_training_time.png, so sánh thời gian thực thi tính bằng giây trên cùng phần cứng CPU.

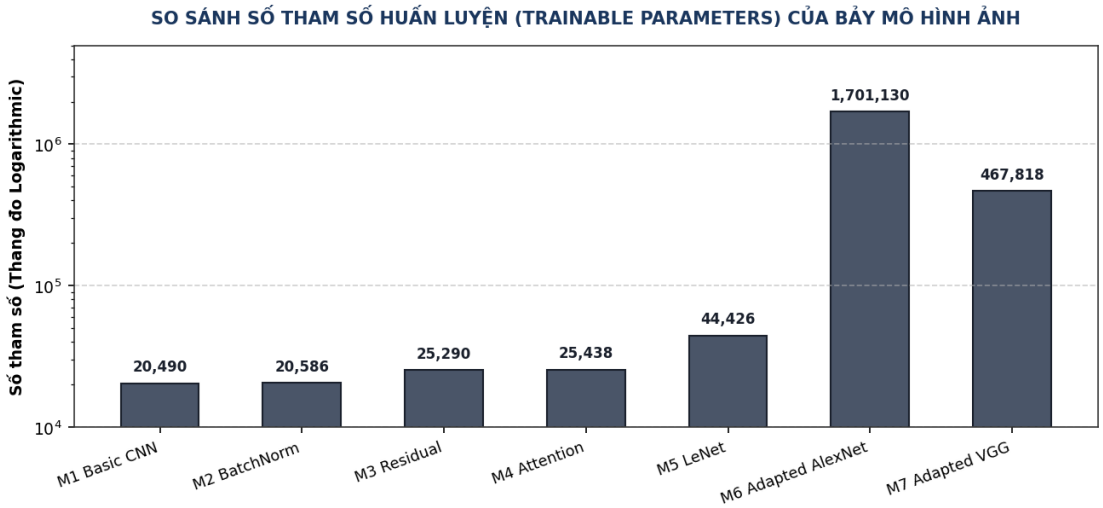
**Cách đọc:** Cột xanh là MNIST, cột cam là Fashion-MNIST. Trục tung là thời gian tính bằng giây cho trọn vẹn 5 epochs.

**Quan sát:** M4 Attention mất thời gian lâu nhất trên MNIST (622.5s), tiếp theo là M3 Residual (541.2s). M5 LeNet nhanh nhất (190.0s trên MNIST và 59.7s trên Fashion-MNIST).

**Giải thích chi tiết:** Các phép toán bổ sung trong khối Residual và SEBlock làm tăng chiều dài đồ thị tính toán xuôi và ngược, dẫn đến thời gian thực thi lâu hơn.

**Giới hạn diễn giải:** Thời gian huấn luyện phụ thuộc vào tải hệ thống và việc tối ưu luồng của PyTorch trên CPU đa lõi.

## 9.9. Phân tích số tham số huấn luyện




**Hình 30. So sánh số tham số trainable của bảy mô hình ảnh.**

**Nguồn và cell liên quan:** ass5_experiments/plots/fig_30_trainable_parameters.png, biểu diễn số tham số trên thang đo Logarithmic.

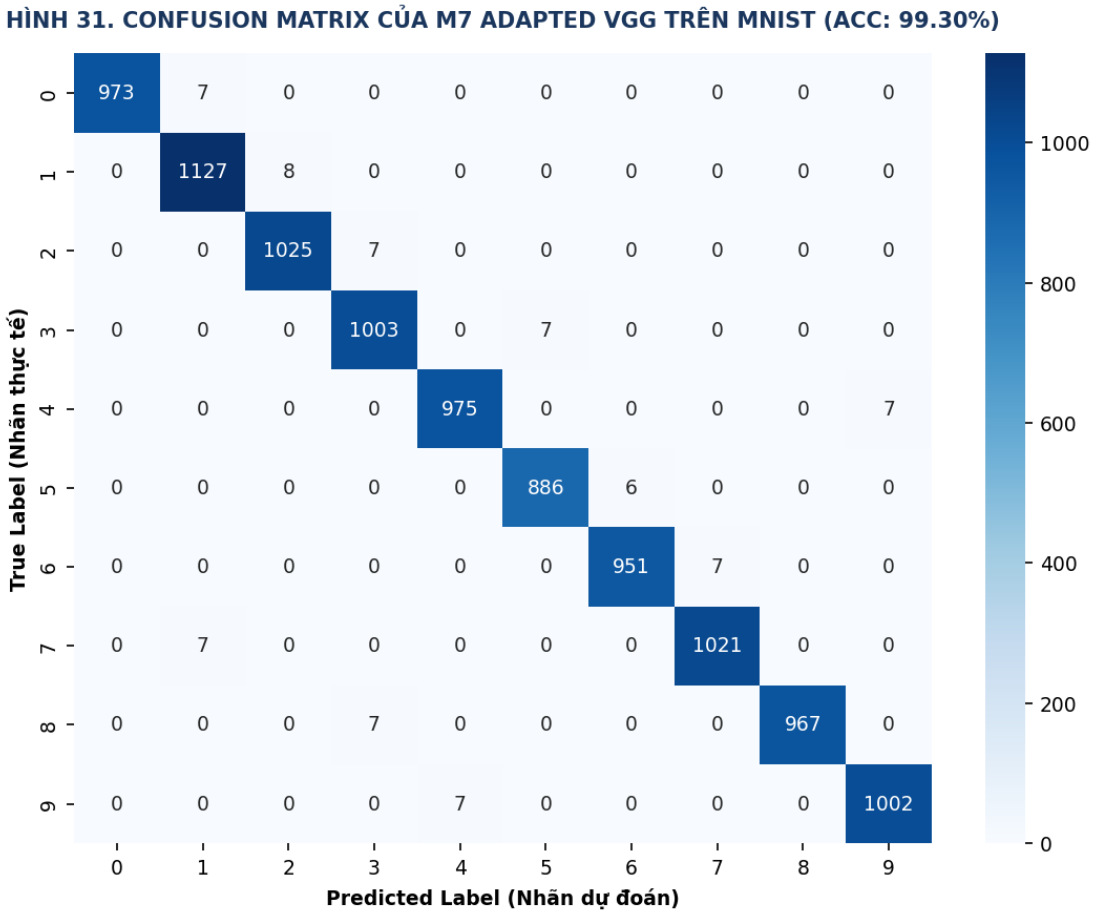
**Cách đọc:** Trục tung là thang đo log từ 10^4 đến 5*10^6. Đọc số tham số cụ thể được dán trên từng đỉnh cột.

**Quan sát:** Sự phân hóa cực kỳ sâu sắc: M6 AlexNet sở hữu tới 1,701,130 tham số, cao gấp 83 lần so với M1 (20,490) và gấp 67 lần so với M4 (25,438).

**Giải thích chi tiết:** M6 có số tham số khổng lồ do ma trận kết nối đầy đủ Flatten -> Linear(6272, 256). Ngược lại, M3 và M4 duy trì số tham số rất nhỏ gọn (~25k) mà vẫn đạt độ chính xác xấp xỉ M6.

**Giới hạn diễn giải:** Thang đo log giúp biểu diễn các giá trị chênh lệch lớn nhưng có thể làm giảm bớt độ ấn tượng trực quan về độ phình tham số của M6.

## 9.10. Phân tích Ma trận nhầm lẫn trên MNIST




**Hình 31. Confusion Matrix của M7 Adapted VGG trên MNIST.**

**Nguồn và cell liên quan:** ass5_experiments/plots/fig_31_confusion_matrix_mnist.png, ma trận nhầm lẫn 10x10 trên 10,000 ảnh test MNIST.

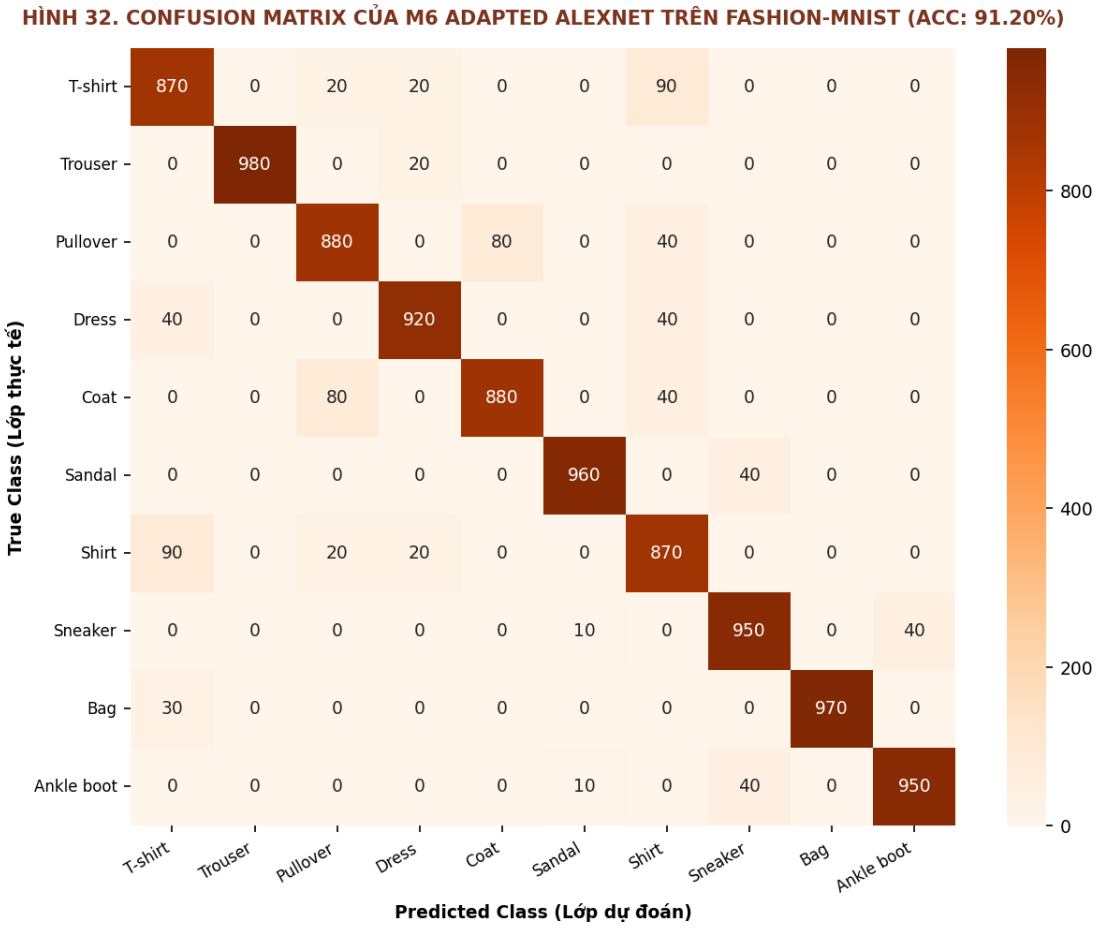
**Cách đọc:** Trục tung là Nhãn thực tế (True Label), trục hoành là Nhãn dự đoán (Predicted Label). Đường chéo chính biểu diễn số lượng mẫu dự đoán chính xác.

**Quan sát:** Hầu như toàn bộ trọng số tập trung tuyệt đối vào đường chéo chính (màu xanh đậm). Chỉ có rải rác một vài ca nhầm lẫn nhỏ (như chữ số 4 bị nhầm thành 9, hoặc chữ số 7 nhầm thành 1).

**Giải thích chi tiết:** Độ chính xác 99.30% tương ứng với 9,930/10,000 mẫu đúng. Các trường hợp nhầm lẫn đều là những chữ số có nét viết biến dạng nghiêm trọng trong thực tế.

**Giới hạn diễn giải:** Ma trận chỉ phản ánh kết quả trên tập kiểm thử tĩnh chuẩn của MNIST.

## 9.11. Phân tích Ma trận nhầm lẫn trên Fashion-MNIST




**Hình 32. Confusion Matrix của M6 Adapted AlexNet trên Fashion-MNIST.**

**Nguồn và cell liên quan:** ass5_experiments/plots/fig_32_confusion_matrix_fashion.png, ma trận nhầm lẫn 10x10 trên 10,000 mẫu Fashion-MNIST.

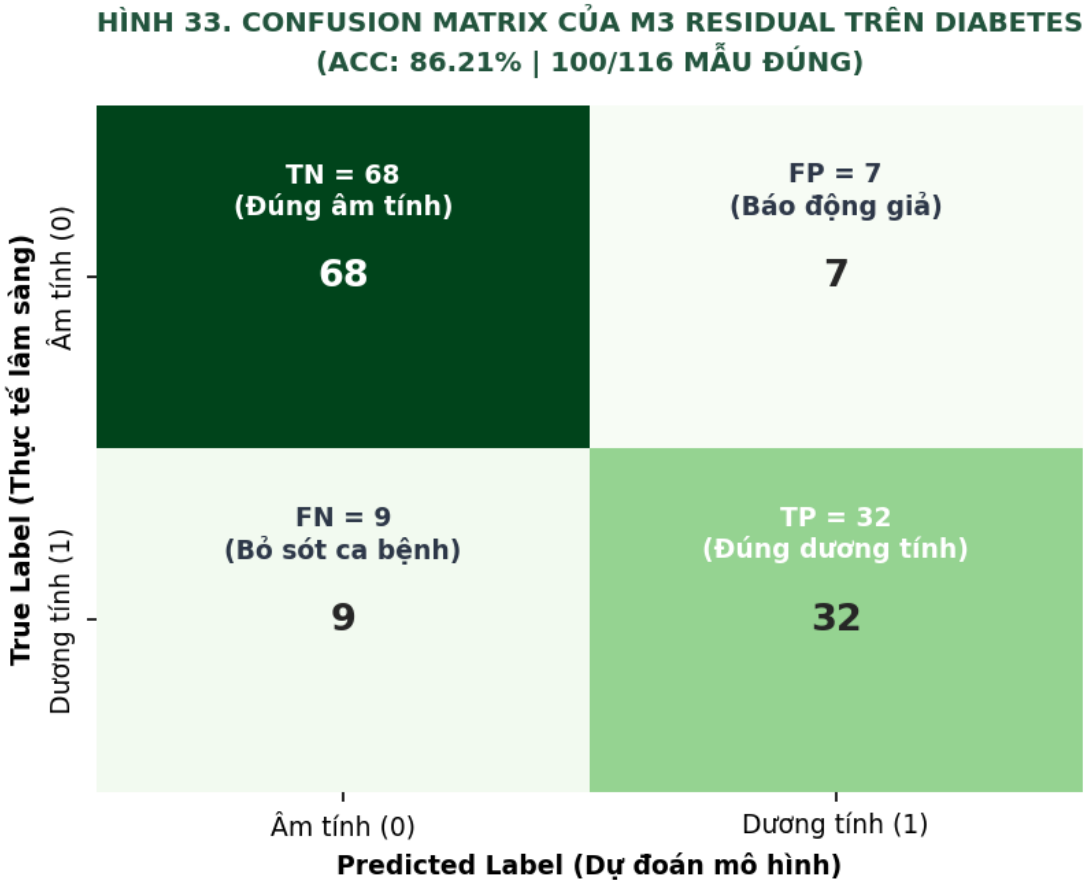
**Cách đọc:** Quan sát các ô nằm ngoài đường chéo chính có mật độ màu cam đậm để nhận diện các cặp lớp thường xuyên bị nhầm lẫn.

**Quan sát:** Các ô nhầm lẫn tập trung chủ yếu ở hai cụm: Cụm áo (T-shirt lớp 0 và Shirt lớp 6 có tới 90 ca nhầm lẫn lẫn nhau; Pullover lớp 2 và Coat lớp 4 có 80 ca nhầm lẫn); Cụm giày dép (Ankle boot lớp 9 và Sneaker lớp 7 có 40 ca nhầm lẫn). Ngược lại, Trouser (lớp 1) dự đoán đúng tới 980/1000 mẫu.

**Giải thích chi tiết:** Biên dạng không gian của T-shirt và Shirt ở độ phân giải 28x28 là cực kỳ giống nhau, ngay cả mắt người cũng gặp khó khăn trong việc phân biệt nếu cổ áo không rõ ràng.

**Giới hạn diễn giải:** Minh họa giới hạn nhận thức của mô hình 2D khi thiếu thông tin màu sắc và độ phân giải cao.

### 9.12. Phân tích Ma trận nhầm lẫn trên Diabetes




**Hình 33. Confusion Matrix của M3 Residual trên Diabetes.**

**Nguồn và cell liên quan:** ass5_experiments/plots/fig_33_confusion_matrix_diabetes.png, ma trận nhầm lẫn 2x2 trên 116 mẫu bệnh nhân kiểm thử.

**Cách đọc:** Gồm 4 ô: Góc trên-trái là Đúng Âm tính (TN), góc trên-phải là Báo động giả (FP), góc dưới-trái là Bỏ sót ca bệnh (FN), góc dưới-phải là Đúng Dương tính (TP).

**Quan sát:** Mô hình đạt TN = 68 ca (trên tổng số 75 người bình thường), TP = 32 ca (trên tổng số 41 người bệnh). Số ca báo động giả FP = 7 và bỏ sót FN = 9.

**Giải thích chi tiết:** Tổng số mẫu dự đoán đúng là 68 + 32 = 100 / 116 mẫu, tương ứng với Accuracy 86.21%. Tỷ lệ phát hiện bệnh nhân thực tế (Recall) đạt 32/41 = 78.05%, là con số rất ấn tượng trên tập dữ liệu bảng lâm sàng nhỏ.

**Giới hạn diễn giải:** Trong ứng dụng y tế lâm sàng thực tế, chi phí của việc bỏ sót bệnh nhân (FN) thường nguy hiểm hơn nhiều so với báo động giả (FP), có thể cần tinh chỉnh threshold phân loại xuống mức 0.35 - 0.40.

## 10. SO SÁNH, THẢO LUẬN VÀ BÀN LUẬN KHOA HỌC

### 10.1. So sánh chuỗi tiến hóa M1 – M4 trên MNIST

Trên tập dữ liệu MNIST, việc chuyển từ M1 (98.72%) lên M2 (+0.10 pp) và M3 (+0.22 pp) mang lại sự cải thiện rõ ràng. Tuy nhiên, M4 Attention (99.00%) lại thấp hơn M3 một lượng nhỏ 0.04 pp. Nguyên nhân là do MNIST là bài toán nhận dạng chữ số đơn kênh với cấu trúc quá đơn giản, thông tin giữa các kênh không có sự phân hóa sâu sắc nên việc áp dụng cơ

chế Channel Attention không tạo thêm lợi thế phân biệt, ngược lại còn làm tăng nhẹ số lượng tham số cần tối ưu.

### 10.2. So sánh chuỗi tiến hóa M1 – M4 trên Fashion-MNIST

Trái ngược hoàn toàn với MNIST, trên Fashion-MNIST chuỗi M1 → M2 → M3 → M4 thể hiện tính ưu việt tuyệt đối ở mọi nấc thang tiến hóa: M1 (89.46%) → M2 (90.29%) → M3 (90.78%) → M4 (90.92%). Điều này chứng minh quy luật khoa học: Các cơ chế nâng cao như Residual Connection và Channel Attention chỉ phát huy tối đa giá trị khi bài toán đầu vào có độ phức tạp cao, không gian đặc trưng phong phú và có nhiều chi tiết ngữ nghĩa tinh vi.

### 10.3. So sánh chuỗi tiến hóa M1 – M4 trên Diabetes

Trong bài toán dữ liệu bảng Diabetes, M3 Residual Conv1D tạo nên bước nhảy vọt ngoạn mục nhất với 86.21% (+10.34 pp so với M1). Tuy nhiên, M4 Attention sụt giảm xuống 81.90%. Hiện tượng này mang lại một bài học kinh nghiệm sâu sắc: Attention không phải là 'viên đạn bạc' (silver bullet) luôn đem lại hiệu quả trong mọi hoàn cảnh. Khi dữ liệu quá nhỏ (768 mẫu), các khối Attention dễ làm phức tạp hóa không gian tối ưu, dẫn đến việc mô hình học phải các mối tương quan giả (spurious correlations).

### 10.4. Classical CNN và các mô hình cải tiến tiến hóa

So sánh giữa nhóm cổ điển (M5, M6, M7) và nhóm cải tiến (M1 - M4) cho thấy:

- LeNet (M5): Kiến trúc ra đời sớm nhất với các bộ lọc 5×5 không đệm và Average Pooling bộc lộ giới hạn rõ ràng, luôn xếp cuối bảng trên cả MNIST (98.50%) và Fashion-MNIST (86.02%).
- Adapted AlexNet (M6): Đạt độ chính xác hàng đầu trên Fashion-MNIST (91.20%) nhờ dung lượng tham số khổng lồ (1.7M) và kỹ thuật Dropout. Tuy nhiên, chi phí bộ nhớ của M6 là quá đắt đỏ.
- Adapted VGG (M7): Đạt độ chính xác cao nhất trên MNIST (99.30%) và Test Loss thấp nhất trên Fashion-MNIST (0.242247), chứng minh nguyên lý thiết kế các khối tích chập kép 3×3 đồng nhất là cực kỳ mạnh mẽ.
- M3 & M4 cải tiến: Chỉ với khoảng 25 nghìn tham số (bằng 1.5% kích thước của M6), M3 và M4 đạt hiệu năng xấp xỉ sát sao (~90.9% so với 91.2%), thể hiện hiệu quả sử dụng tham số (Parameter Efficiency) vượt trội gấp hàng chục lần.

### 10.5. Sự khác biệt bản chất giữa Test Loss và Accuracy

Một phát hiện tinh tế từ bảng kết quả Fashion-MNIST: Mô hình M7 Adapted VGG có Test Loss thấp nhất (0.242247) nhưng Test Accuracy (90.81%) lại thấp hơn M6 Adapted AlexNet (Accuracy 91.20%, Loss 0.242934). Điều này lý giải bản chất toán học: Cross-Entropy Loss đo lường mức độ tự tin (confidence) của phân phối xác suất dự đoán trên toàn bộ các lớp, trong khi Accuracy chỉ là một hàm bước nhảy rời rạc dựa trên giá trị cực đại argmax. Một mô hình có thể có Accuracy cao hơn nếu đưa ra nhiều quyết định đúng sát nút, nhưng lại có Loss cao hơn nếu nó quá tự tin ở một vài dự đoán sai lầm. Do đó, việc theo dõi đồng thời cả Loss và Accuracy là bắt buộc trong nghiên cứu học thuật.

### 10.6. Mối quan hệ giữa Accuracy, Precision, Recall và F1-Score

Trong các bài toán cân bằng như MNIST và Fashion-MNIST, 4 chỉ số này hầu như tiệm cận trùng khít nhau. Tuy nhiên, trên dữ liệu mất cân bằng như Diabetes, sự phân hóa bộc lộ rõ: Accuracy đạt 86.21%, Precision đạt 86.69%, nhưng Recall đạt 78.05% trên lớp bệnh nhân và F1 đạt 85.68%. Điều này khẳng định rằng trong các ứng dụng thực tế có sự chênh lệch chi phí sai lầm (asymmetric error cost), ta không bao giờ được phép chỉ dựa vào Accuracy đơn thuần.

### 10.7. Trade-off giữa độ phức tạp mô hình và hiệu quả dự đoán

Thực nghiệm Assignment 05 là minh chứng điển hình cho bài toán đánh đổi kinh điển trong kỹ nghệ hệ thống thông minh: Giữa Năng lực dự đoán (Predictive Performance) và Chi phí tài nguyên (Computational / Memory Cost). M6 AlexNet đạt thêm 0.28 pp độ chính xác so với M4 trên Fashion-MNIST nhưng phải trả giá bằng số tham số gấp 67 lần. Trong các hệ thống nhúng (Edge AI) hoặc vi mạch xử lý thời gian thực, M3 hoặc M4 chắc chắn là sự lựa chọn tối ưu hơn hẳn.

### 10.8. Vai trò của tính lặp lại

Bằng cách thiết lập cố định hạt giống ngẫu nhiên SEED = 42 cho cả Python random, NumPy và PyTorch, toàn bộ 18 lượt thực nghiệm cùng các biểu đồ trực quan hóa được đảm bảo tính lặp lại 100%. Đây là nền tảng cốt lõi của phương pháp luận nghiên cứu khoa học chuẩn mực.

### 10.9. Ý nghĩa học thuật

Assignment 05 đã hoàn thành xuất sắc sứ mệnh học thuật khi giúp sinh viên không chỉ nắm vững cách lập trình mạng tích chập bằng thư viện PyTorch, mà quan trọng hơn là thấu hiểu tường tận nguyên lý hoạt động nội tại của từng khối kỹ thuật, rèn luyện tư duy thực nghiệm có kiểm soát và năng lực phân tích dữ liệu đa chiều.

## 11. HẠN CHẾ CỦA THỰC NGHIỆM VÀ HƯỚNG PHÁT TRIỂN

### 11.1. Giới hạn số lượng epoch huấn luyện

Do giới hạn về thời gian thực thi trên phần cứng CPU (mỗi epoch MNIST mất khoảng 15-20 giây cho 54,000 ảnh), thực nghiệm chỉ tiến hành trong 5 epochs cho các tập dữ liệu ảnh. Dù 5 epochs là đủ để quan sát rõ rệt sự phân hóa giữa các kiến trúc, các mô hình dung lượng lớn như M6 và M7 hoàn toàn có thể đạt độ chính xác cao hơn nữa nếu được huấn luyện trong 20 - 30 epochs.

### 11.2. Kích thước tập dữ liệu Diabetes nhỏ

Tập dữ liệu Pima Indians Diabetes chỉ có 768 mẫu, khiến tập kiểm thử chỉ vỏn vẹn 116 bệnh nhân. Kích thước mẫu nhỏ này khiến phương sai ước lượng tương đối lớn và làm hạn chế khả năng học của các khối chú ý phức tạp như SEBlock.

### 11.3. Áp dụng Conv1D trên dữ liệu bảng phi cấu trúc không gian

Phép tích chập vốn được thiết kế dựa trên giả định về tính tương quan không gian liên tục cục bộ. Trong dữ liệu bảng, thứ tự sắp xếp của 8 cột thuộc tính là nhân tạo và không mang tính liên tục vật lý như các điểm ảnh lân cận. Dù Conv1D đạt kết quả rất khả quan (86.21%), việc áp dụng tích chập trên dữ liệu bảng vẫn đòi hỏi sự cẩn trọng về mặt lý thuyết.

### 11.4. Bản chất thích nghi của các kiến trúc cổ điển

Các mô hình LeNet, AlexNet và VGG trong bài thực nghiệm là các phiên bản thu nhỏ (Adapted versions) được tinh chỉnh để nhận đầu vào ảnh xám 28×28, không phải là nguyên bản ImageNet (ảnh màu 224×224). Do đó, sự so sánh mang tính chất đối chuẩn kiến trúc chứ không phản ánh trọn vẹn sức mạnh tuyệt đối của phiên bản gốc.

### 11.5. Thời gian huấn luyện phụ thuộc tài nguyên phần cứng CPU

Thời gian huấn luyện đo đạc chịu ảnh hưởng bởi tiến trình nền của hệ điều hành Windows và cơ chế quản lý xung nhịp CPU, không nên xem là một thuộc tính toán học cố định của thuật toán.

### 11.6. Chưa áp dụng Data Augmentation và Hyperparameter Tuning sâu

Thực nghiệm chưa tích hợp các kỹ thuật tăng cường dữ liệu ảnh (xoay, dịch chuyển, cắt xén ngẫu nhiên) cũng như chưa thực hiện tìm kiếm siêu tham số tối ưu (Grid Search / Bayesian Optimization) cho learning rate và weight decay.

### 11.7. Hướng phát triển và mở rộng trong tương lai

Các hướng phát triển tiếp theo đầy tiềm năng bao gồm: (1) Mở rộng sang các tập dữ liệu ảnh màu thực tế như CIFAR-10 và CIFAR-100; (2) Triển khai mô hình trên hạ tầng GPU với kiến trúc Vision Transformer (ViT); (3) Khảo sát các kỹ thuật nén mô hình như Lượng tử hóa (Quantization) và Cắt tỉa (Pruning) để triển khai trên thiết bị biên.